This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

PPSF: A Privacy-Preserving and Secure Framework using Blockchain-based Machine-Learning for IoT-driven Smart Cities

Prabhat Kumar, *Student Member, IEEE,* Randhir Kumar, *Student Member, IEEE,* Gautam Srivastava, *Senior Member, IEEE,* Govind P. Gupta, *Member, IEEE,*, Rakesh Tripathi, *Senior Member, IEEE,* Thippa Reddy Gadekallu, Neal N. Xiong, *Senior Member, IEEE.*

***Abstract*—With the evolution of the Internet of Things (IoT), smart cities have become the mainstream of urbanization. IoT networks allow distributed smart devices to collect and process data within smart city infrastructure using an open channel, the Internet. Thus, challenges such as centralization, security, privacy (e.g., performing data poisoning and inference attacks), transparency, scalability, and verifiability limits faster adaptations of smart cities. Motivated by the aforementioned discussions, we present a Privacy-Preserving and Secure Framework (PPSF) for IoT-driven smart cities. The proposed PPSF is based on two key mechanisms: a two-level privacy scheme and an intrusion detection scheme. First, in a two-level privacy scheme, a blockchain module is designed to securely transmit the IoT data and Principal Component Analysis (PCA) technique is applied to transform raw IoT information into a new shape. In the intrusion detection scheme, a Gradient Boosting Anomaly Detector (GBAD) is applied for training and evaluating the proposed two-level privacy scheme based on two IoT network datasets, namely ToN-IoT and BoT-IoT. We also suggest a blockchain-InterPlanetary File System (IPFS) integrated Fog-Cloud architecture to deploy the proposed PPSF framework. Experimental results demonstrate the superiority of the PPSF framework over some recent approaches in blockchain and non-blockchain systems.**

***Index Terms*—Fog-Cloud architecture, Intelligent Blockchain, Internet of Things, Intrusion Detection System, Machine Learning, Privacy-Preservation**

I. Introduction

BY 2050, more than 70% of the world's population will live in urban areas. This rapid growth in urbanization will create a variety of social, economic, technical, and organizational problems [1]. To efficiently handle the growth of urbanization, several countries have introduced the idea of smart cities. In general, the term 'smart city' refers to the use of technology-based approaches, such as the IoT, big data analysis, cyber-physical systems, and real-time control, to enhance and provide a comfortable life to citizens [2].

The Internet of Things (IoT) is recognized as a key enabling technology that brings/connects all distributed sensors and smart devices together, to collect and process data within smart city infrastructure using an open channel i.e., Internet [3]. Smart city application heavily relies on the efficient utilization of data generated by these sensors/devices and includes a wide range of areas such as health care, energy management, transport, water and irrigation management.

In smart cities, various sensors are linked/connected to the Internet. This leads to consequent challenges associated with security and privacy. First, security concerns are linked with IoT sensor inter-connectivity, which poses several possible threats to smart cities targeting IoT devices. Smart City attacks are usually of two kinds, physical and/or cyber-attacks. In a physical attack, the adversary is close to the IoT sensors, and can easily modify/temper the IoT devices and sensors used for communication. This attack includes Permanent Denial of Service (DoS), malicious code injection, and fake node injection. In cyber-attacks, the adversary attempts to insert malware or malicious software to gain unauthorized access to smart city network components. Eavesdropping, Theft, Ransomware, Backdoor, Denial-of-Service (DoS), and Distributed DoS (DDoS) are included in these attacks. Secondly, the privacy issue involves compromising sensitive information by performing data poisoning and inference attacks. In data poisoning and inference attacks, an attacker tries to manipulate the data obtained from a smart object by adding fictitious measurements of data [4]. This has detrimental impacts on intrusion detection and data analytic systems built on machine learning [5]. Also, these attacks can lead to wasteful use of IoT device power and can interrupt regular communications among devices. Thus, ensuring security and privacy are the key requirement in smart city and their network.

The purpose of IoT is to deploy smart devices and sensors that can provide a pervasive environment and ubiquitous experience. The key challenge in IoT deployment is providing transparent and seamless services with security and privacy. To ensure IoT security, various intrusion detection/prevention systems (IDSs/IPSs) using statistical, and machine learning techniques have been developed [6]. However, most of the existing conventional security approaches use a cloud-based centralized deployment to meet the computational require-

Prabhat Kumar, Randhir Kumar, Govind Gupta, and Rakesh Tripathi are all with the National Institute of Technology, Raipur, India. Email: pkumar.phd2019.it@nitrr.ac.in, rkumar.phd2018.it@nitrr.ac.in, gpgupta.it@nitrr.ac.in, rtripathi.it@nitrr.ac.in

Gautam Srivastava is with Department of Mathematics and Computer Science, Brandon University, Brandon, MB R7A 6A9, Canada and Research Center for Interneural Computing, China Medical University, Taichung 40402, Taiwan. Email: srivastavag@brandonu.ca

Thippa Reddy G. is with Vellore Institute of Technology, India. Email: thippareddy.g@vit.ac.in

Neal N. Xiong is with the Department of Mathematics and Computer Science, Northeastern State University, OK, USA. Email: xiong31@nsuok.edu
Corresponding Author: Gautam Srivastava (srivastavag@brandonu.ca)

ments. Cloud-based deployment has several limitations, such as geo-distribution, low mobility support, single point of failure, huge idle energy consumption, high congestion, and high latency [7]. A functional provenance is used by the cloud datacenter to denote when, where, and how data is stored, accessed, updated, and deleted in the cloud datacenter. However, provenance records are prone to tampering with by a threat agent, as a result, the provenance system can be disabled or misused. Another challenge in a centralized setting is that it lacks traceability, transparency, and is governed by multi-parties [8].

One potential solution is to use a fog computing architecture, where the computational resources are transferred and processed close to the users/devices. Fog computing for IoT enables several advantages such as it provides a path for data offloading, reduces the communication latency, and increases the spectral efficiency [9]. Additionally, edge devices are responsible for gathering and processing the obtained data in fog. These edge-processed data are sent to participating IoT devices based on their specifications or aggregated and forwarded to the cloud network to further processes and/or for long-term storage [10]. IoT-fog integration, however, opens new challenges of scalability, verifiability, security and privacy of data generated by IoT devices [11].

Blockchain has recently generated new interest due to its decentralized infrastructure and transparent communication among system entities [12]–[15]. The underlying technology consists of various features like immutability, integrity, and a tamper-resistant ledger. Thus, adopting blockchain in IoT-driven smart cities can ensure better security and privacy of the data. To address the aforementioned challenges about IoT-driven smart cities, blockchain technology needs to get thoroughly investigated. Blockchain is a decentralized and trustworthy distributed system [16]. It facilitates auditability, immutability, data transparency, trust among distributed IoT nodes, and can trace tamper-resistant historical records in the ledger [17]. The storage of large data volumes is expensive for blockchain, however, storing data hashes in the ledger, instead of the data itself, is effective [18]. In consideration of the constrained storage space of each blockchain node, this paper uses InterPlanetary File System (IPFS), a distributed file system that is content-addressable and stores data with high integrity and durability. In IPFS, there is no central server and data is replicated and stored across the Internet in various IPFS nodes [19]. For each uploaded file, IPFS allocates a unique hash. In blockchain-IPFS integration, for each transaction, actual data is stored in IPFS and the returned unique hash is uploaded in block [20]. Therefore, an intelligent blockchain-IPFS integrated Fog-Cloud architecture can be a suitable deployment framework.

A. Motivation and Key Challenges

To the best of our knowledge, a smart city network architecture using blockchain and machine learning is limited in the literature. There are a variety of issues to be tackled (see also Section II). Below, the main challenges are listed:

1) Constructing a privacy-preservation mechanism that can efficiently protect confidential data from disclosure by unauthorized users, during processing it over smart city network is challenging task [1], [2], [16], [19].
2) Designing an adaptable intrusion detector that can proficiently distinguish normal from abnormal observations in a high-speed smart city network is challenging. Such networks are associated to the interconnectivity of IoT sensors and devices, located at multiple location [1]–[3], [6], [21].
3) Developing a decentralized architecture that is extremely scalable, verifiable (i.e., supports audit-trail using timestamp records), and transparent in IoT-driven smart city network is a challenging issue. Achieving above requirements using current Fog-Cloud architecture is difficult, [7]–[9], [9].
4) Ensuring the framework efficiency is measured by actual IoT-based datasets consisting of real-world IoT-based attacks. Such datasets are the main criteria during the training process to acquire model knowledge [6], [16], [21].

B. Key Contributions

To enhance the privacy and security of IoT data, we propose PPSF and an intelligent blockchain framework that integrates blockchain and PCA-based transformation and machine learning approaches. The main contributions of this paper can be summarized as follows:

1) We propose a two-level privacy-preservation scheme. The first level involves a blockchain and an enhanced Proof of Work (ePoW) approach based on smart contracts using the Ethereum network. This level authenticates records of IoT data and prevents the modification of original data through data poisoning attacks. The second level uses a transformation technique based on Principal Component Analysis (PCA) that transforms authenticated IoT data into a new encoded format and avoids inference attack data that could be learned from system-based machine learning.
2) A Gradient Boosting Anomaly Detector (GBAD), based on LightGBM utility system is designed and applied before and after the two-level privacy-preservation scheme, to evaluate the capability of the proposed PPSF framework in discovering various cyber-attacks in the smart city network.
3) A decentralized deployment solution by integrating blockchain-IPFS with existing fog-cloud architecture is presented. This deployment strategy mitigates the limitations of current fog-cloud or stand-alone cloud-based deployments.
4) The proposed framework and its methods are assessed using two IoT network datasets ToN-IoT and BoT-IoT, and the systems' performance is compared with some latest state-of-the-art peer-privacy preserving techniques to determine its effectiveness

The remainder of this paper is arranged as follows. Section II presents a brief discussion of various privacy-preservation, intrusion detection approaches, and existing relevant background and related work. In Section III, we present our pro-

posed framework with its deployment architecture respectively. In Section IV, datasets, evaluation criteria, experimental results, and comparison with recent state-of-the-art frameworks are discussed. Finally, Section V concludes this paper with future work.

II. Related Work

We review concepts and previous studies, particularly those focusing on privacy preservation including blockchain, smart contracts, and intrusion detection techniques in IoT and their networks.

A. Threat models in IoT-driven Smart cities

In PPSF, we consider the Dolev-Yao (DY) threat model for ensuring security in the proposed authentication of IoT devices and their communications within a smart city. As the adversary can control the entire system and can modify (poisoning and inference attack) the information. The adversary can also guess the private key of the respective public key for IoT devices. The adversary can violet the security by session key disclosure. Thus, session key creation of two different IoT devices must have a dependency on temporal and permanent secrets.

B. Privacy-Preserving Techniques

Privacy-preservation can be described as a process of securing private confidential data transactions from disclosure by unauthorized malicious parties [8]. Privacy-preservation techniques can be of five types; encryption-based, authentication-based, differential privacy, perturbation-based and blockchain-based approaches [4], [22], [23]. The perturbation technique transforms original data into a new format to protect the confidentiality of the data using various transformation approaches, such as statistical and data projection measures [4].

The blockchain-based privacy-preservation technique applies the concept of blockchain technology. Blockchain consists of distributed increasing lists of transaction records, called blocks, which are connected and protected through cryptographic approaches. Blockchain facilitates a peer-to-peer (P2P) protocol-based infrastructure that can sustain single point failure [11]. The consensus methodology implements a common, unmistakable order of transactions and block creation by providing integrity, and consistency in the blockchain network across geographically dispersed nodes. By architecture, blockchain facilitates characteristics viz., decentralization, transparency, and auditability.

Three popular consensus techniques, namely, Proof-of-Work (PoW), Proof-of-Stake (PoS), and Practical Byzantine Fault Tolerance (PBFT) have been broadly applied in blockchain applications, to authenticate the legitimacy of a transaction [23]. The PoW technique selects a node in a distributed network to authenticate and create a new block in each round of consensus consuming a predefined amount of computational resources. During computation, the participating nodes need to solve a cryptographic puzzle. The node which first solves the puzzle has the right to add a new block in the ledger. However, PoW gets violated, if the malicious computing node controls more than 51% of the total computational resources in the network, this is known as the 51% attack [4]. On the other hand, in PoS, the creator of the block are completely determined by their stake value in accounts rather than a huge resource and computational power. Although, the mining node still needs to solve a cryptographic puzzle, however, the node does not need to adjust nonce many times, rather the puzzle is solved based on the number of stake [3]. The third consensus method is PBFT, which comprises five different phases namely, pre-prepare phase, prepare phase, commit phase, and reply phase. The PBFT method maintains a common state and takes a consistent action in each round of consensus computation. PBFT is energy-efficient and implemented on hyperledger fabric [24].

The blockchain serves as a platform for smart contracts to be hosted and executed on. Smart contracts are self-executed programs that run across the peer nodes of a blockchain network and verifies the transaction's business logic with certain conditions [3]. The working principle of smart contracts is broadly divided into two parts namely value and state. The states of the contracts are preset according to the conditional statement such as "If-Then" statements [11]. The smart contracts are agreed and signed by all the participating nodes and submitted as transactions in the network, and then transactions are disseminated via peer-to-peer (P2P) network, confirmed by the miners and securely stored in the blockchain network. The contracts' address further gets accessible to all the parties, to invoke the contracts' functionality by sending a transaction. Miners are full participating nodes in the network who verify the transactions by contributing their computational resources [16]. More precisely, after the miners obtain a contract for the creation or initiation of a transaction, they execute the code of the specific contract in their EVM-enabled local Sandboxed Execution Environment (SEE). Then the smart contracts match the scenario of the triggering condition concerning the input transactions initiated by the participating nodes [25], [26]. If the conditions are met successfully then response actions are executed, then transactions get validated by miners, and it gets recorded to a new block [3]. Finally, the new block is chained into the blockchain network after reaching the consensus.

C. Intrusion Detection Techniques

An intrusion is an act that is conducted knowingly or illegally in a networking system that indicates variation from the systems' normal activities [19]. An Intrusion Detection System (IDS) is a combination of tools, resources and methods to monitor the traffic of the network and protect the permissible users from abnormal activities that try to compromise the availability, confidentiality, and integrity of the information system. IDS is mainly divided on the basis of: i) position; and ii) detection approach [5]. First, IDS based on the position in network architecture can be viewed as: i) host-based IDS (HIDS); and ii) network-based IDS (NIDS). HIDS is a component of the software located on the monitoring system which usually tracks a host machine. This gives HIDS excellent insight into the state of the system but at the cost of poor isolation from the network. Moreover, an attacker

can mislead or disable the HIDS, if it obtains access to the system [6]. NIDS are normally physically distinct tools, situated on the "upstream" network of the system being tracked, and on a typical network, they usually monitor several separate networks. These programmes, though, have little to no information available about the internal status of the systems they control, which may make it more difficult to identify them [16]. Second, detection-based IDS can be of two types: i) signature or knowledge-based (SIDS); and ii) anomaly or behaviour-based (AIDS). Signature-based IDS matches the stored pattern (string) or combination of patterns of known threats with the detected threat to identify intrusions. However, SIDS has a high detection rate and fails to detect new malicious attacks [4]. In AIDS normal user profile is derived by monitoring network connection and regular activities of legitimate users. When the deviation of behaviours between ongoing activities and the derived model is observed, then that variation is interpreted as an intrusion [23].

Machine learning (ML) is an artificial intelligence approach that is widely used in designing IDS. ML techniques can process massive datasets and have sufficient generalizability to detect different attack vectors [2]. Another advantage of using ML techniques is that it uses few parameters and can easily be updated in real-time after its deployment in smart city [5]. However, the security system can degrade due to massive data processing and analysis. LightGBM is one of the ensemble techniques that integrate multiple decision trees (DTs) into a strong classifier using the boosting approach i.e., Gradient Boosting Decision Tree (GBDT). This approach aims to improve computational efficiency and has been effective in solving big datasets efficiently in real-time deployment [27].

D. Related Studies

In recent years, many studies using blockchain and machine learning are used to design privacy-preservation and intrusion detection techniques. For instance, Sharma et al. [28] presented a distributed smart city architecture using Software Defined Networking (SDN) and blockchain technology. Keshk *et al.* [29] proposed a cyber-physical systems (CPS) privacy preservation mechanism. For feature transformation, independent component analysis (ICA), and intrusion detection, Decision Tree (DT), Support Vector Machine (SVM), and Naive Bayes (NB) are applied.

Rathore *et al.* [30] reviewed a decentralized security mechanism using blockchain, SDN, Fog, and mobile edge computing for IoT networks. A memory-hardened PoW is applied to provide security and privacy. Liang *et al.* [31] presented a hybrid placement strategy for IDS based on a multi-agent system. A deep neural network (DNN) is used to detect the anomaly and agents' actions are stored on a blockchain. The proposed method is, however, tested using obsolete CPS datasets. Keshk *et al.* [32] presented an IDS based on a privacy-preserving approach. Pre-processing of features combined with Gaussian mixture is used to protect privacy, and the Kalman filter technique is applied to detect intrusions. A two-level privacy-preserving paradigm is proposed by Keshk *et al.* [4]. In the privacy module blockchain and Variational AutoEncoder were used and the Long Short Term Memory technique for detecting intrusions. Alkadi *et al.* [23] utilized blockchain and smart contracts to authenticate IoT records and the Bidirectional LSTM (BiLSTM) technique to design collaborative IDS in cloud networks.

Shafiq *et al.* [33] designed an IDS in IoT-based smart city. This technique is based on a bijective soft set combined with the CorrACC method for feature selection. The performance is evaluated using various ML approaches. Similarly, Shafiq *et al.* [34] proposed a feature selection technique using CorrAUC integrated with TOPSIS and Shannon Entropy based on a bijective soft set. Hasan *et al.* [35] designed an IDS for IoT sensors. The authors evaluated their model based on different ML techniques, to detect intrusions, and the result shows improved performance using random forest (RF). Shafiq *et al.* [36] designed a bijective soft approach to select the best classification technique based on ML models. In the evaluation phase, different ML approaches such as RF, NB, C4.5 and SVM are used. Table I and Table II summarizes existing works related to blockchain and non-blockchain respectively.

Table I: Comparative analysis of PPSF and non-blockchain related frameworks

Authors	Year	A	B	C	D	E	F	G	H	I	J
Keshk *et al.* [29]	2018	×	×	✓	✓	×	✓	×	×	×	×
Hasan *et al.* [35]	2019	×	×	✓	✓	×	×	×	×	×	×
Koroniotis *et al.* [37]	2019	×	×	✓	✓	×	×	×	×	×	×
Shafiq *et al.* [33]	2020	×	×	✓	✓	×	×	×	×	×	×
Haider *et al.* [38]	2020	×	×	✓	✓	×	×	×	×	×	×
Shafiq *et al.* [34]	2020	×	×	✓	✓	×	×	×	×	×	×
Soe *et al.* [39]	2020	×	×	✓	✓	×	×	×	×	×	×
Shafiq *et al.* [36]	2020	×	×	✓	✓	×	×	×	×	×	×
Proposed (PPSF)	2020	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

A: Ledger Distribution; B: Verifiability; C: IDS; D: Security; E: Privacy; F: Transparency; G: Integrity; H: Scalability; I: Offchain; J: Decentralized.

Table II: Comparative analysis of PPSF and blockchain related frameworks

Authors	Year	A	B	C	D	E	F	G	H	I	J
Sharma *et al.* [28]	2018	✓	×	×	✓	✓	×	×	✓	×	✓
Keshk *et al.* [32]	2019	×	×	✓	✓	×	✓	×	×	×	✓
Rathore *et al.* [30]	2019	✓	×	×	✓	×	✓	×	✓	×	✓
Chao *et al.* [31]	2019	✓	✓	✓	✓	×	✓	×	×	×	✓
Zhang *et al.* [24]	2020	✓	✓	×	✓	✓	✓	✓	×	×	✓
Arachchige *et al.* [22]	2020	✓	✓	×	✓	✓	✓	✓	✓	✓	✓
Alkadi *et al.* [23]	2020	✓	✓	✓	✓	×	✓	×	×	×	✓
Proposed (PPSF)	2020	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

A: Ledger Distribution; B: Verifiability; C: IDS; D: Security; E: Privacy; F: Transparency; G: Integrity; H: Scalability; I: Offchain; J: Decentralized.

III. Our Proposed PPSF framework

The components of the proposed PPSF are presented in this section. This includes the two-level scheme of privacy protection and detection of intrusion. Fig. (1) illustrates the framework itself. Each component is described briefly below:

A. Two-level privacy-preservation mechanism

i) ***Privacy using Blockchain and Smart contracts:*** This section explains different entities and their working association in the proposed framework. There are different phases includes in the framework like initialization, registration, and

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

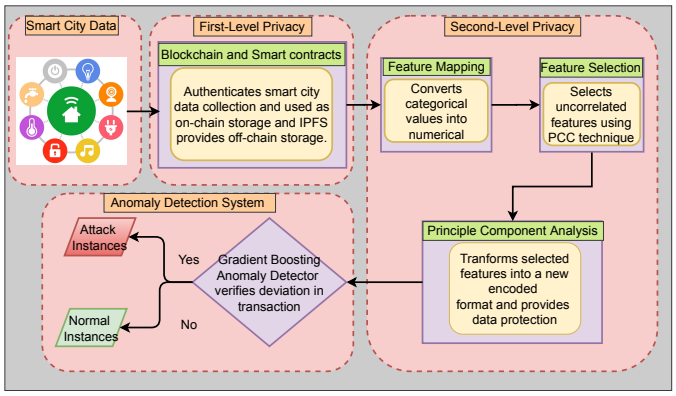


Figure 1: Proposed privacy-preserving secure framework (PPSF) using blockchain-based machine-learning for IoT-driven smart cities

authentication of the proposed model entity. Next, block creation, block updates, and dynamic IoT node addition are presented. Further, encryption and decryption of sensor data are presented. The complete process of all phases is described below.

(i) Initialization Phase

The trusted party ($\mathbb{TP}$) completes this phase, to begin the entire framework by performing the following steps.

Step-1: The trusted party ($\mathbb{TP}$) picks the large prime number $\mathbb{P}_n$, for non-singular elliptic curve $\mathbb{E}_{\mathbb{P}_n}(r,s)$: $v^2 = w^3 + rv + (s \bmod \mathbb{P}_n)$ over finite field $\mathbb{Z}_{\mathbb{P}_n}$, where $\mathbb{Z}_{\mathbb{P}_n} = \{ 0,1,2, \ldots, P_n - 1\}$ with initial point $\mathbb{Q}$ over $\mathbb{E}_{\mathbb{P}_n}(r,s)$, where the order must be large as $\mathbb{P}_n$ on infinity point $\mathbb{I}_f$ with hash function $H_f(.)$, where $H_f(.)$ denotes one-way cryptographic hash function.

Step-2: The $\mathbb{TP}$ picks a random private key $\mathbb{TP}_{prkey} \in \mathbb{Z}^*_{\mathbb{P}_n}$ and finds respective public key $\mathbb{TP}_{pbkey}$ using multiplication of elliptic curve point $\mathbb{TP}_{pbkey} = \mathbb{TP}_{prkey}$ x $\mathbb{Q}$, and makes secret of $\mathbb{TP}_{prkey}$ and distribute the $\mathbb{TP}_{pbkey}$ and $H_f(.)$.

(ii) Registration Phase

This phase explores registration process of the various entities of the proposed model.

- IoT Device ($\mathbb{IOTD}$) Registration: In this phase the $\mathbb{TP}$ makes registration of IoT device ($\mathbb{IOTD}$) in offline mode. The steps involved in the process of registration are described below.

  STEP-1 : The $\mathbb{TP}$ sets identity of IoT devices ($\mathbb{ID}_{\mathbb{IOTD}}$) and takes a temporary identity ($\mathbb{TMID}_{\mathbb{IOTD}}$) , and picks a random secret ($\mathbb{R}_s1$) $\in \mathbb{Z}^*_{\mathbb{P}_n}$, to determine the pseudo random identity ($\mathbb{PRID}_{\mathbb{IOTD}}$) of the $\mathbb{IOTD}$ using $\mathbb{PRID}_{\mathbb{IOTD}}$ = $H_f(\mathbb{ID}_{\mathbb{IOTD}} \,||\, \mathbb{R}_s1 \,||\, \mathbb{MKY}_{\mathbb{TP}})$, where $\mathbb{MKY}_{\mathbb{TP}}$ specifies master key of the trusted party ($\mathbb{TP}$).

  STEP-2 : The $\mathbb{TP}$ sets random private key ($\mathbb{IOTD}_{prkey}$) $\in \mathbb{Z}^*_{\mathbb{P}_n}$ and evaluates public key ($\mathbb{IOTD}_{pbkey}$) = $\mathbb{IOTD}_{prkey}$ x $\mathbb{Q}$, and evaluates the temporary credential ($\mathbb{TPC}_{\mathbb{IOTD}}$) = H($\mathbb{PRID}_{\mathbb{IOTD}} \,||\, \mathbb{IOTD}_{prkey} \,||\, \mathbb{MKY}_{\mathbb{TP}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}}$), where $\mathbb{TSTP}_{\mathbb{IOTD}}$ specifies the timestamp of $\mathbb{IOTD}$ registration.

  STEP-3 : The $\mathbb{TP}$ initialize all the essential credential for IoT device ($\mathbb{IOTD}$) using $\{$ ($\mathbb{PRID}_{\mathbb{IOTD}}$, $\mathbb{TMID}_{\mathbb{IOTD}}$, $\mathbb{TPC}_{\mathbb{IOTD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$, ($\mathbb{IOTD}_{prkey}$, $\mathbb{IOTD}_{pbkey}$) $\}$. Further, $\mathbb{TP}$ distribute the $\mathbb{IOTD}_{pbkey}$ key of respective IoT devices ($\mathbb{IOTD}$) for future use. The summary of IoT devices registration is shown in Table III.

Table III: Summary of IoT device registration process

Trusted Party ($\mathbb{TP}$)	**IoT device** ($\mathbb{IOTD}$)
$\mathbb{TP}$ picks $\mathbb{ID}_{\mathbb{IOTD}}$, $\mathbb{TMID}_{\mathbb{IOTD}}$, $\mathbb{R}_s1 \in \mathbb{Z}^*_{\mathbb{P}_n}$ Find $\mathbb{PRID}_{\mathbb{IOTD}} = H_f(\mathbb{ID}_{\mathbb{IOTD}} \,||\, \mathbb{R}_s1 \,||\, \mathbb{MKY}_{\mathbb{TP}})$ $\mathbb{TP}$ sets $\mathbb{IOTD}_{prkey} \in \mathbb{Z}^*_{\mathbb{P}_n}$ Evaluates $\mathbb{IOTD}_{pbkey} = \mathbb{IOTD}_{prkey}$ x $\mathbb{Q}$ Finds $(\mathbb{TPC}_{\mathbb{IOTD}}) = H_f(\mathbb{PRID}_{\mathbb{IOTD}} \,||\, \mathbb{IOTD}_{prkey}$ $||\, \mathbb{MKY}_{\mathbb{TP}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}})$ sets $\mathbb{IOTD}$ with {($\mathbb{PRID}_{\mathbb{IOTD}}$, $\mathbb{TMID}_{\mathbb{IOTD}}$, $\mathbb{TPC}_{\mathbb{IOTD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$, ($\mathbb{IOTD}_{prkey}$, $\mathbb{IOTD}_{pbkey}$) $\}$	Stores {($\mathbb{PRID}_{\mathbb{IOTD}}$, $\mathbb{TMID}_{\mathbb{IOTD}}$, $\mathbb{TPC}_{\mathbb{IOTD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$, ($\mathbb{IOTD}_{prkey}$, $\mathbb{IOTD}_{pbkey}$) $\}$ inside memory for future use.

- Fog and Cloud Registration: In this phase fog ($\mathbb{FOG}$) and cloud ($\mathbb{CLOUD}$) are registered in offline mode by a trusted party ($\mathbb{TP}$). The process of IoT devices ($\mathbb{IOTD}$) connections with $\mathbb{FOG}$ and $\mathbb{FOG}$ connections with $\mathbb{CLOUD}$ is outlined at the deployment stage. The $\mathbb{FOG}$ receives data from the registered $\mathbb{IOTD}$ and $\mathbb{CLOUD}$ receives data from registered $\mathbb{FOG}$. The registration process of $\mathbb{FOG}$ is discussed below.

  STEP-1 : The $\mathbb{TP}$ picks identity of fog ($\mathbb{ID}_{\mathbb{FOG}}$) and sets temporary identity ($\mathbb{TMID}_{\mathbb{FOG}}$) , and picks the random secret ($\mathbb{F}_r1$) $\in \mathbb{Z}^*_{\mathbb{P}_n}$, to find the pseudo random identity ($\mathbb{PRID}_{\mathbb{FOG}}$) of the $\mathbb{FOG}$ using $\mathbb{PRID}_{\mathbb{FOG}} = H_f(\mathbb{ID}_{\mathbb{FOG}} \,||\, \mathbb{F}_r1 \,||\, \mathbb{MKY}_{\mathbb{TP}}, \mathbb{TSTP}_{\mathbb{FOG}})$, where $\mathbb{MKEY}_{\mathbb{TA}}$ denotes the master key of the trusted authority ($\mathbb{TA}$) and $\mathbb{TSTP}_{\mathbb{FOG}}$ denotes registration timestamp of $\mathbb{ID}_{\mathbb{FOG}}$.

  STEP-2 : The $\mathbb{TP}$ sets all required credential for the FOG ($\mathbb{FOG}$) using $\{$ ($\mathbb{PRID}_{\mathbb{FOG}}$, $\mathbb{TMID}_{\mathbb{FOG}}$, $\mathbb{TPC}_{\mathbb{FOG}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$ $\}$. Next, $\mathbb{FOG}$ picks a random private key ($\mathbb{FOG}_{prkey}$) $\in \mathbb{Z}^*_{\mathbb{P}_n}$ and finds respective public key ($\mathbb{FOG}_{pbkey}$) = $\mathbb{FOG}_{prkey}$ x $\mathbb{Q}$. Further, $\mathbb{FOG}$ stores the ($\mathbb{FOG}_{prkey}$, $\mathbb{FOG}_{pbkey}$) to its immutable database with details as $\{$ ($\mathbb{PRID}_{\mathbb{FOG}}$, $\mathbb{TMID}_{\mathbb{FOG}}$, $\mathbb{TPC}_{\mathbb{FOG}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$, ($\mathbb{FOG}_{prkey}$, $\mathbb{FOG}_{pbkey}$) $\}$ and distribute the $\mathbb{FOG}_{pbkey}$ key for further use. The summary of FOG registration is discussed in Table IV.

Table IV: Summary of Fog registration process

Trusted Party ($\mathbb{TP}$)	**FOG** ($\mathbb{FOG}$)
$\mathbb{TP}$ selects $\mathbb{ID}_{\mathbb{FOG}}$, $\mathbb{TMID}_{\mathbb{FOG}}$, $\mathbb{F}_r1 \in \mathbb{Z}^*_{\mathbb{P}_n}$ $\mathbb{PRID}_{\mathbb{FOG}} = H_f(\mathbb{ID}_{\mathbb{FOG}}$ $||\, \mathbb{F}_r1 \,||\, \mathbb{MKY}_{\mathbb{TP}}$, $\mathbb{TSTP}_{\mathbb{FOG}})$ $\mathbb{FOG}$ selects $\mathbb{FOG}_{prkey} \in \mathbb{Z}^*_{\mathbb{P}_n}$ Compute $\mathbb{FOG}_{pbkey} = \mathbb{FOG}_{prkey}$ x $\mathbb{Q}$ Initialize with $\{$ ($\mathbb{PRID}_{\mathbb{FOG}}$, $\mathbb{TMID}_{\mathbb{FOG}}$, $\mathbb{TPC}_{\mathbb{FOG}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$ $\}$	Stores $\{$ ($\mathbb{PRID}_{\mathbb{FOG}}$, $\mathbb{TMID}_{\mathbb{FOG}}$, $\mathbb{TPC}_{\mathbb{FOG}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}(r,s)$, $\mathbb{Q}$ $\}$ inside memory for future use.

The registration process of cloud ($\mathbb{CLOUD}$) is discussed below.

STEP-1 : The $\mathbb{TP}$ selects real identity of cloud

($\mathbb{ID}_{\mathbb{CLOUD}}$) and picks temporary identity ($\mathbb{TMID}_{\mathbb{CLOUD}}$) , and chooses the random secret ($\mathbb{C}_r1$) $\in \mathbb{Z}^*_{\mathbb{PN}}$, to find the pseudo random identity ($\mathbb{PRID}_{\mathbb{CLOUD}}$) of the $\mathbb{CLOUD}$ using $\mathbb{PRID}_{\mathbb{CLOUD}} = H_f(\mathbb{ID}_{\mathbb{CLOUD}} \,||\, \mathbb{C}_r1 \,||\, \mathbb{MKY}_{\mathbb{TP}}, \mathbb{TSTP}_{\mathbb{CLOUD}})$, where $\mathbb{MKY}_{\mathbb{TP}}$ denotes the master key of the trusted party ($\mathbb{TP}$) and $\mathbb{TSTP}_{\mathbb{CLOUD}}$ denotes registration timestamp of $\mathbb{ID}_{\mathbb{CLOUD}}$.

STEP-2 : The $\mathbb{TP}$ initializes all the required credential for the cloud ($\mathbb{CLOUD}$) with { ($\mathbb{PRID}_{\mathbb{CLOUD}}$, $\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{TPC}_{\mathbb{CLOUD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$ }. Next, $\mathbb{CLOUD}$ selects random private key ($\mathbb{CLOUD}_{prkey}$) from the finite field $\mathbb{Z}^*_{\mathbb{P}_n}$ and finds respective public key ($\mathbb{CLOUD}_{pbkey}$) = $\mathbb{CLOUD}_{prkey}$ x $\mathbb{Q}$. Further, $\mathbb{CLOUD}$ stores the ($\mathbb{CLOUD}_{prkey}$, $\mathbb{CLOUD}_{pbkey}$) to its immutable database with details as { ($\mathbb{PRID}_{\mathbb{CLOUD}}$, $\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{TPC}_{\mathbb{CLOUD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$, ($\mathbb{CLOUD}_{prkey}$, $\mathbb{CLOUD}_{pbkey}$) } and distribute the $\mathbb{CLOUD}_{pbkey}$ key for further use. The summary of cloud ($\mathbb{CLOUD}$) registraion is discussed in Table V.

Table V: Summary of Cloud registration process

Trusted Party ($\mathbb{TP}$)	**Cloud** ($\mathbb{CLOUD}$)
$\mathbb{TP}$ selects $\mathbb{ID}_{\mathbb{CLOUD}}$, $\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{C}_r1 \in \mathbb{Z}^*_{\mathbb{P}_n}$ $\mathbb{PRID}_{\mathbb{CLOUD}} = H_f(\mathbb{ID}_{\mathbb{CLOUD}}$ $\|\| \mathbb{C}_r1 \|\| \mathbb{MKY}_{\mathbb{TP}}, \mathbb{TSTP}_{\mathbb{CLOUD}})$ $\mathbb{CLOUD}$ selects $\mathbb{CLOUD}_{prkey} \in \mathbb{Z}^*_{\mathbb{P}_n}$ Compute $\mathbb{CLOUD}_{pbkey} = \mathbb{CLOUD}_{prkey}$ x $\mathbb{Q}$ Intialize with { ($\mathbb{PRID}_{\mathbb{CLOUD}}$, $\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{TPC}_{\mathbb{CLOUD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$ }	
	Stores { ($\mathbb{PRID}_{\mathbb{CLOUD}}$, $\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{TPC}_{\mathbb{CLOUD}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$ } inside memory for future use.

(iii) Authentication Phase

This phase discusses authentications of different entities involved in the proposed model operations such as IoT-to-IoT, IoT-to-fog, and Fog-to-Cloud authentication. The detailed operation of the authentication process is discussed below.

- IoT to IoT Authentication Phase: An IoT device $\mathbb{IOTD}_1$ transfer data with another IoT device $\mathbb{IOTD}_2$ securely. As the IoT device is registered by a trusted party $\mathbb{TP}$, the stored secret information gets used for the authentication, where the session key is used for mutual authentication between IoT devices.
  STEP-1: The $\mathbb{IOTD}_1$ establishes operations by choosing secret number within the infinite set $\mathbb{Z}^*_{\mathbb{P}_n}$ and receives timestamp $\mathbb{TSTP}_{\mathbb{IOTD}_1}$ and finds $\mathbb{mIOTD}_1$= H($\mathbb{PRID}_{\mathbb{IOTD}_1}$ $\|\|$ $\mathbb{TMID}_{\mathbb{IOTD}_1}$ $\|\|$ $\mathbb{TPC}_{\mathbb{IOTD}_1}$ $\|\|$ $\mathbb{IOTD}_{1\mathbb{prkey}}$ $\|\|$ $\mathbb{TSTP}_{\mathbb{IOTD}_1}$ $\|\|$ $\mathbb{R}_{s1}$). Further, signature is generated by $\mathbb{IOTD}_1$ i.e. $\mathbb{IOTD}_{1_{\mathbb{sign1}}}$= $\mathbb{mIOTD}_1 \oplus H_f(\mathbb{TMID}_{\mathbb{IOTD}_1} \,||\, \mathbb{IOTD}_{1\mathbb{prkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_1})$ * $\mathbb{IOTD}_{1\mathbb{prkey}}$ (mod $\mathbb{Q}$). Now $\mathbb{IOTD}_1$ sends the message to $\mathbb{IOTD}_2$ using secure channel, i.e. $\mathbb{IOTD}_{1_{\mathbb{msg}}}$ = ($\mathbb{TMID}_{\mathbb{IOTD}_1}$ $\|\|$ $\mathbb{IOTD}_{1_{\mathbb{sign1}}}$ $\|\|$ $\mathbb{TSTP}_{\mathbb{IOTD}_1}$ $\|\|$ $\mathbb{IOTD}_{1\mathbb{pbkey}}$).
  STEP-2: $\mathbb{IOTD}_2$ accepts the message of $\mathbb{IOTD}_{1_{\mathbb{msg}}}$ and checks signature $\mathbb{IOTD}_{1_{\mathbb{sign1}}}$ by $\mathbb{IOTD}_{1_{\mathbb{sign1}}}$ * $\mathbb{Q}$ = $H_f(\mathbb{TMID}_{\mathbb{IOTD}_1} \,||\, \mathbb{IOTD}_{1\mathbb{pbkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_1})$ and also verify the timestamp $\mathbb{TSTP}^*_{\mathbb{IOTD}_1}$ with $|\mathbb{TSTP}^*_{\mathbb{IOTD}_1} - \mathbb{TSTP}_{\mathbb{IOTD}_1}|$ leq $\delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully then $\mathbb{IOTD}_2$ generates random secret $\mathbb{R}_{s2} \in \mathbb{Z}^*_{\mathbb{P}_n}$ and add current timestamp ($\mathbb{TSTP}_{\mathbb{IOTD}_2}$) and compute $\mathbb{mIOTD}_2$= $H_f(\mathbb{PRID}_{\mathbb{IOTD}_2} \,||\, \mathbb{TMID}_{\mathbb{IOTD}_2} \,||\, \mathbb{TPC}_{\mathbb{IOTD}_2} \,||\, \mathbb{IOTD}_{2\mathbb{prkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_2} \,||\, \mathbb{R}_{s2})$, and creates session key ($\mathbb{IOTD}_{2_{\mathbb{SESKEY}}}$) using $\mathbb{mIOTD}_2$ * $\mathbb{Q}$. Next, $\mathbb{IOTD}_2$ creates signatures $\mathbb{IOTD}_{2_{\mathbb{sign2}}}$= $\mathbb{mIOTD}_2 \oplus H_f(\mathbb{TMID}_{\mathbb{IOTD}_2} \,||\, \mathbb{TMID}_{\mathbb{IOTD}_1} \,||\, \mathbb{IOTD}_{2\mathbb{pbkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_2})$ * $\mathbb{IOTD}_{2\mathbb{prkey}}$ (mod $\mathbb{Q}$). Next, $\mathbb{IOTD}_2$ sends message $\mathbb{IOTD}_{2_{\mathbb{msg}}}$ = ($\mathbb{TMID}_{\mathbb{IOTD}_2}$, $\mathbb{IOTD}_{2_{\mathbb{SESKEY}}}$, $\mathbb{IOTD}_{2_{\mathbb{sign2}}}$, $\mathbb{TSTP}_{\mathbb{IOTD}_2}$, $\mathbb{IOTD}_{2\mathbb{pbkey}}$) to $\mathbb{IOTD}_1$ using secure channel.
  STEP-3: $\mathbb{IOTD}_1$ accepts the message and checks the session key $\mathbb{IOTD}_{2_{\mathbb{SESKEY}}}$ using $\mathbb{mIOTD}_1$ * $\mathbb{IOTD}_{2_{\mathbb{SESKEY}}}$, and also verifies the timestamp $\mathbb{TSTP}^*_{\mathbb{IOTD}_2}$ with $|\mathbb{TSTP}^*_{\mathbb{IOTD}_2} - \mathbb{TSTP}_{\mathbb{IOTD}_2}|$ leq $\delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully, then it verifies the signature $\mathbb{IOTD}_{2_{\mathbb{sign2}}}$ by $\mathbb{IOTD}_{2_{\mathbb{sign2}}}$ * $\mathbb{Q}$ = $H_f(\mathbb{TMID}_{\mathbb{IOTD}_2} \,||\, \mathbb{TMID}_{\mathbb{IOTD}_1} \,||\, \mathbb{IOTD}_{2\mathbb{pbkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_2} \,||\, \mathbb{IOTD}_{2_{\mathbb{SESKEY}}})$. If the signature matches successful then $\mathbb{IOTD}_1$ creates new timestamp $\mathbb{TSTP}_{\mathbb{IOTD}_{\mathbb{New1}}}$ and compute $\mathbb{mIOTD}_{New1}$ = $H_f(\mathbb{IOTD}_{2_{\mathbb{SESKEY}}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_{\mathbb{New1}}})$ and sends acknowledgement $\mathbb{IOTD}_{1_{\mathbb{ACKW}}}$= $\mathbb{IOTD}_{2_{\mathbb{SESKEY}}}$, $\mathbb{TSTP}_{\mathbb{IOTD}_{\mathbb{New1}}}$ to $\mathbb{IOTD}_2$ using secure channel.
  STEP-4: $\mathbb{IOTD}_2$ accepts the message and checks the session key $\mathbb{IOTD}_{2_{\mathbb{SESKEY}}}$ using $H_f(\mathbb{IOTD}_{2_{\mathbb{SESKEY}}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}_{\mathbb{New1}}})$, and also verifies the timestamp $\mathbb{TSTP}^*_{\mathbb{IOTD}_{\mathbb{New1}}}$ with $|\mathbb{TSTP}^*_{\mathbb{IOTD}_{\mathbb{New1}}} - \mathbb{TSTP}_{\mathbb{IOTD}_{\mathbb{New1}}}|$ leq $\delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches, then both $\mathbb{IOTD}_1$ and $\mathbb{IOTD}_2$ shares the secret key $\mathbb{IOTD}_{1_{\mathbb{SKEY}}}$ = $\mathbb{IOTD}_{2_{\mathbb{SKEY}}}$ for further communication. The detailed authentication process of IoTD-to-IoTD is described in Table VI.
- IoT to FOG Authentication Phase: In this phase, IoT device $\mathbb{IOTD}$ and $\mathbb{FOG}$ authentication is described. The stored details in memory of $\mathbb{IOTD}$ and $\mathbb{FOG}$ are used for authentication, and the session key is established to secure authentication. The process of authentication is discussed below.

  STEP-1: The $\mathbb{IOTD}$ begins the activities of authentication by selecting secret number $\in \mathbb{Z}^*_{\mathbb{P}_n}$ and sets timestamp $\mathbb{TSTP}_{\mathbb{IOTD}}$ and finds $\mathbb{mIOTD}$= $H_f(\mathbb{PRID}_{\mathbb{IOTD}} \,||\, \mathbb{TMID}_{\mathbb{IOTD}} \,||\, \mathbb{TPC}_{\mathbb{IOTD}} \,||\, \mathbb{IOTD}_{\mathbb{prkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}} \,||\, \mathbb{R}_{s1})$. Next, signature is generated by $\mathbb{IOTD}$ i.e. $\mathbb{IOTD}_{\mathbb{sign1}}$= $\mathbb{mIOTD} \oplus H_f(\mathbb{TMID}_{\mathbb{IOTD}} \,||\, \mathbb{IOTD}_{\mathbb{prkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}})$ * $\mathbb{IOTD}_{\mathbb{prkey}}$ (mod $\mathbb{Q}$). Now $\mathbb{IOTD}$ sends the message to $\mathbb{FOG}$ using secure channel, i.e. $\mathbb{IOTD}_{\mathbb{msg}}$ = ($\mathbb{TMID}_{\mathbb{IOTD}}$ $\|\|$ $\mathbb{IOTD}_{\mathbb{sign1}}$ $\|\|$ $\mathbb{TSTP}_{\mathbb{IOTD}}$ $\|\|$ $\mathbb{IOTD}_{\mathbb{pbkey}}$).
  STEP-2: $\mathbb{FOG}$ accepts the message of $\mathbb{IOTD}_{\mathbb{msg}}$ and verify signature $\mathbb{IOTD}_{\mathbb{sign1}}$ by $\mathbb{IOTD}_{\mathbb{sign1}}$ * $\mathbb{Q}$ = $H_f(\mathbb{TMID}_{\mathbb{IOTD}} \,||\, \mathbb{IOTD}_{\mathbb{pbkey}} \,||\, \mathbb{TSTP}_{\mathbb{IOTD}})$ and also verify timestamp $\mathbb{TSTP}^*_{\mathbb{IOTD}}$ with $|\mathbb{TSTP}^*_{\mathbb{IOTD}} - \mathbb{TSTP}_{\mathbb{IOTD}}|$ leq $\delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully, then $\mathbb{FOG}$ creates random secret $\mathbb{F}_s1 \in \mathbb{Z}^*_{\mathbb{P}_n}$ and also add current timestamp ($\mathbb{TSTP}_{\mathbb{FOG}}$) and compute $\mathbb{mFOG}$= $H_f(\mathbb{PRID}_{\mathbb{FOG}} \,||\, \mathbb{TMID}_{\mathbb{FOG}} \,||\, \mathbb{TPC}_{\mathbb{FOG}} \,||\, \mathbb{FOG}_{\mathbb{prkey}} \,||\, \mathbb{TSTP}_{\mathbb{FOG}} \,||\, \mathbb{F}_{r1})$, and creates

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

Table VI: IoT-to-IoT Authentication

IoT device ($\mathtt{IOTD}_1$)	**IoT device** ($\mathtt{IOTD}_2$)
Select secret number ($\mathbb{R}_{s1}$) $\in \mathbb{Z}^*_{P_n}$ Find $\mathtt{mIOTD}_1 = H_f(\mathtt{PRID}_{\mathtt{IOTD}_1} \|\| \mathtt{TMID}_{\mathtt{IOTD}_1} \|\| \mathtt{TPC}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{1\mathtt{prkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_1} \|\| \mathbb{R}_{s1})$ Compute signature, $\mathtt{IOTD}_{1\mathtt{sign1}} = \mathtt{mIOTD}_1 \oplus H_f(\mathtt{TMID}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{1\mathtt{prkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_1}) * \mathtt{IOTD}_{1\mathtt{prkey}} \pmod{\mathbb{Q}}$ Send Message, $\mathtt{IOTD}_{1\mathtt{msg}} = (\mathtt{TMID}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{1\mathtt{sign1}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{1\mathtt{pbkey}})$ to $\mathtt{IOTD}_2$	
	verify signature $\mathtt{IOTD}_{1\mathtt{sign1}}$ by $\mathtt{IOTD}_{1\mathtt{sign1}} * \mathbb{Q} = H_f(\mathtt{TMID}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{1\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_1})$ and also check timestamp $\mathtt{TSTP}^*_{\mathtt{IOTD}_1}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{IOTD}_1} - \mathtt{TSTP}_{\mathtt{IOTD}_1} \rvert \; leq \; \delta\mathbb{T}$ Creates session key ($\mathtt{IOTD}_{2\mathtt{SESKEY}}$) using $\mathtt{mIOTD}_2 * \mathbb{Q}$ Create signature $\mathtt{IOTD}_{2\mathtt{sign2}} = \mathtt{mIOTD}_2 \oplus H_f(\mathtt{TMID}_{\mathtt{IOTD}_2} \|\| \mathtt{TMID}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{2\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_2}) * \mathtt{IOTD}_{2\mathtt{prkey}} \pmod{\mathbb{Q}}$ Send message $\mathtt{IOTD}_{2\mathtt{msg}} = (\mathtt{TMID}_{\mathtt{IOTD}_2}, \mathtt{IOTD}_{2\mathtt{SESKEY}}, \mathtt{IOTD}_{2\mathtt{sign2}}, \mathtt{TSTP}_{\mathtt{IOTD}_2}, \mathtt{IOTD}_{2\mathtt{pbkey}})$ to $\mathtt{IOTD}_1$ using secure channel
checks the session key $\mathtt{IOTD}_{2\mathtt{SESKEY}}$ using $\mathtt{mIOTD}_1 * \mathtt{IOTD}_{2\mathtt{SESKEY}}$, and also verifies the timestamp $\mathtt{TSTP}^*_{\mathtt{IOTD}_2}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{IOTD}_2} - \mathtt{TSTP}_{\mathtt{IOTD}_2} \rvert \; leq \; \delta\mathbb{T}$, where δ Verify signature $\mathtt{IOTD}_{2\mathtt{sign2}}$ by $\mathtt{IOTD}_{2\mathtt{sign2}} * \mathbb{Q} = H_f(\mathtt{TMID}_{\mathtt{IOTD}_2} \|\| \mathtt{TMID}_{\mathtt{IOTD}_1} \|\| \mathtt{IOTD}_{2\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_2} \|\| \mathtt{IOTD}_{2\mathtt{SESKEY}}$ Create new timestamp $\mathtt{TSTP}_{\mathtt{IOTD}_{New1}}$ and compute $\mathtt{mIOTD}_{New1} = H_f(\mathtt{IOTD}_{2\mathtt{SESKEY}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_{New1}})$ Send acknowledgement $\mathtt{IOTD}_{1\mathtt{ACKW}} = \mathtt{IOTD}_{2\mathtt{SESKEY}}, \mathtt{TSTP}_{\mathtt{IOTD}_{New1}}$ to $\mathtt{IOTD}_2$ using secure channel	
	Check the session key $\mathtt{IOTD}_{2\mathtt{SESKEY}}$ using $H_f(\mathtt{IOTD}_{2\mathtt{SESKEY}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_{New1}})$, and also verify the timestamp $\mathtt{TSTP}^*_{\mathtt{IOTD}_{New1}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{IOTD}_{New1}} - \mathtt{TSTP}_{\mathtt{IOTD}_{New1}} \rvert \; leq \; \delta\mathbb{T}$ If both the conditions are matches, then both $\mathtt{IOTD}_1$ and $\mathtt{IOTD}_2$ shares the secret key $\mathtt{IOTD}_{1\mathtt{SKEY}} = \mathtt{IOTD}_{2\mathtt{SKEY}}$ for further communication

session key ($\mathtt{FOG}_{\mathtt{SESKEY}}$) using $\mathtt{mFOG} * \mathbb{Q}$. Next, $\mathtt{FOG}$ creates signatures $\mathtt{FOG}_{\mathtt{sign2}} = \mathtt{mFOG} \oplus H_f(\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{FOG}_{\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}}) * \mathtt{FOG}_{\mathtt{prkey}} \pmod{\mathbb{Q}}$. Next, $\mathtt{FOG}$ sends message $\mathtt{FOG}_{\mathtt{msg}} = (\mathtt{TMID}_{\mathtt{FOG}}, \mathtt{FOG}_{\mathtt{SESKEY}}, \mathtt{FOG}_{\mathtt{sign2}}, \mathtt{TSTP}_{\mathtt{FOG}}, \mathtt{FOG}_{\mathtt{pbkey}})$ to $\mathtt{IOTD}$ using secure channel.

STEP-3: $\mathtt{IOTD}$ accepts the message and checks the session key $\mathtt{FOG}_{\mathtt{SESKEY}}$ using $\mathtt{mIOTD} * \mathtt{FOG}_{\mathtt{SESKEY}}$, and also verifies the timestamp $\mathtt{TSTP}^*_{\mathtt{FOG}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{FOG}} - \mathtt{TSTP}_{\mathtt{FOG}} \rvert \; leq \; \delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully, then it verifies the signature $\mathtt{FOG}_{\mathtt{sign2}}$ by $\mathtt{FOG}_{\mathtt{sign2}} * \mathbb{Q} = H_f(\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{FOG}_{\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{SESKEY}})$. If the signature matches successfully, then $\mathtt{IOTD}$ creates new timestamp $\mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}}$ and compute $\mathtt{mIOTD}_{New} = H_f(\mathtt{FOG}_{\mathtt{SESKEY}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}})$ and sends acknowledgement $\mathtt{IOTD}_{\mathtt{ACKW}} = \mathtt{FOG}_{\mathtt{SESKEY}}, \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}}$ to $\mathtt{FOG}$ using secure channel.

STEP-4: $\mathtt{FOG}$ accepts the message and checks the session key $\mathtt{FOG}_{\mathtt{SESKEY}}$ using $H_f(\mathtt{FOG}_{\mathtt{SESKEY}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}})$, and also verifies the timestamp $\mathtt{TSTP}^*_{\mathtt{IOTD}_{\mathtt{New}}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{IOTD}_{\mathtt{New}}} - \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}} \rvert \; leq \; \delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully, then both $\mathtt{IOTD}$ and $\mathtt{FOG}$ shares the secret key $\mathtt{IOTD}_{\mathtt{SKEY}} = \mathtt{FOG}_{\mathtt{SKEY}}$ for further communication. The detailed authentication process of IoTD-to-FOG is described in Table VII.

Table VII: IoT-to-fog Authentication

IoT Device ($\mathtt{IOTD}$)	**FOG** ($\mathtt{FOG}$)
Select secret number ($\mathbb{R}_{s1}$) $\in \mathbb{Z}^*_{P_n}$ Find $\mathtt{mIOTD} = H_f(\mathtt{PRID}_{\mathtt{IOTD}} \|\| \mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{TPC}_{\mathtt{IOTD}} \|\| \mathtt{IOTD}_{\mathtt{prkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}} \|\| \mathbb{R}_{s1})$ Compute signature, $\mathtt{IOTD}_{\mathtt{sign1}} = \mathtt{mIOTD} \oplus H_f(\mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{IOTD}_{\mathtt{prkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}}) * \mathtt{IOTD}_{\mathtt{prkey}} \pmod{\mathbb{Q}}$ Send Message, $\mathtt{IOTD}_{\mathtt{msg}} = (\mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{IOTD}_{\mathtt{sign1}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}} \|\| \mathtt{IOTD}_{\mathtt{pbkey}})$ to $\mathtt{FOG}$	
	Verify signature $\mathtt{IOTD}_{\mathtt{sign1}}$ by $\mathtt{IOTD}_{\mathtt{sign1}} * \mathbb{Q} = H_f(\mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{IOTD}_{\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}})$ and also check timestamp $\mathtt{TSTP}^*_{\mathtt{IOTD}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{IOTD}} - \mathtt{TSTP}_{\mathtt{IOTD}} \rvert \; leq \; \delta\mathbb{T}$ Creates session key ($\mathtt{FOG}_{\mathtt{SESKEY}}$) using $\mathtt{mFOG} * \mathbb{Q}$ Create signature $\mathtt{FOG}_{\mathtt{sign2}} = \mathtt{mFOG} \oplus H_f(\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{FOG}_{\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}}) * \mathtt{FOG}_{\mathtt{prkey}} \pmod{\mathbb{Q}}$ Send message $\mathtt{FOG}_{\mathtt{msg}} = (\mathtt{TMID}_{\mathtt{FOG}}, \mathtt{FOG}_{\mathtt{SESKEY}}, \mathtt{FOG}_{\mathtt{sign2}}, \mathtt{TSTP}_{\mathtt{FOG}}, \mathtt{FOG}_{\mathtt{pbkey}})$ to $\mathtt{IOTD}$ using secure channel
checks the session key $\mathtt{FOG}_{\mathtt{SESKEY}}$ using $\mathtt{mFOG} * \mathtt{FOG}_{\mathtt{SESKEY}}$, and also verifies the timestamp $\mathtt{TSTP}^*_{\mathtt{FOG}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{FOG}} - \mathtt{TSTP}_{\mathtt{FOG}} \rvert \; leq \; \delta\mathbb{T}$, where δ Verify signature $\mathtt{FOG}_{\mathtt{sign2}}$ by $\mathtt{FOG}_{\mathtt{sign2}} * \mathbb{Q} = H_f(\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{TMID}_{\mathtt{IOTD}} \|\| \mathtt{FOG}_{\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{SESKEY}}$ Create new timestamp $\mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}}$ and compute $\mathtt{mIOTD}_{New} = H_f(\mathtt{FOG}_{\mathtt{SESKEY}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}})$ Send acknowledgement $\mathtt{IOTD}_{\mathtt{ACKW}} = \mathtt{FOG}_{\mathtt{SESKEY}}, \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}}$ to $\mathtt{FOG}$ using secure channel	
	Check the session key $\mathtt{FOG}_{\mathtt{SESKEY}}$ using $H_f(\mathtt{FOG}_{\mathtt{SESKEY}} \|\| \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}})$, and also verify the timestamp $\mathtt{TSTP}^*_{\mathtt{IOTD}_{\mathtt{New}}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{IOTD}_{\mathtt{New}}} - \mathtt{TSTP}_{\mathtt{IOTD}_{\mathtt{New}}} \rvert \; leq \; \delta\mathbb{T}$ If both conditions matches, then both $\mathtt{IOTD}$ and $\mathtt{FOG}$ shares the secret key $\mathtt{IOTD}_{\mathtt{SKEY}} = \mathtt{FOG}_{\mathtt{SKEY}}$ for further communication

- Fog to Cloud Authentication Phase: In this phase, fog $\mathtt{FOG}$ and $\mathtt{CLOUD}$ authentication is described. The stored details in memory of $\mathtt{FOG}$ and $\mathtt{CLOUD}$ are used for authentication, and the session key is established to secure authentication. The process of authentication is discussed below.

  STEP-1: The $\mathtt{FOG}$ starts operations by selecting secret number $\in \mathbb{Z}^*_{P_n}$ and creates timestamp $\mathtt{TSTP}_{\mathtt{FOG}}$ and finds $\mathtt{mFOG} = H_f(\mathtt{PRID}_{\mathtt{FOG}} \|\| \mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{TPC}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{prkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}} \|\| \mathbb{R}_{s1})$. Next signature is created by $\mathtt{FOG}$ i.e. $\mathtt{FOG}_{\mathtt{sign1}} = \mathtt{mFOG} \oplus H_f(\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{prkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}}) * \mathtt{FOG}_{\mathtt{prkey}} \pmod{\mathbb{Q}}$. Now $\mathtt{FOG}$ sends the message to $\mathtt{CLOUD}$ using secure channel, i.e. $\mathtt{FOG}_{\mathtt{msg}} = (\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{sign1}} \|\| \mathtt{TSTP}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{pbkey}})$.

  STEP-2: $\mathtt{CLOUD}$ receives the message of $\mathtt{FOG}_{\mathtt{msg}}$ and verify signature $\mathtt{FOG}_{\mathtt{sign1}}$ by $\mathtt{FOG}_{\mathtt{sign1}} * \mathbb{Q} = H_f(\mathtt{TMID}_{\mathtt{FOG}} \|\| \mathtt{FOG}_{\mathtt{pbkey}} \|\| \mathtt{TSTP}_{\mathtt{FOG}})$ and also checks timestamp $\mathtt{TSTP}^*_{\mathtt{FOG}}$ with $\lvert \mathtt{TSTP}^*_{\mathtt{FOG}} - \mathtt{TSTP}_{\mathtt{FOG}} \rvert \; leq \; \delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successful, then $\mathtt{CLOUD}$

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

creates random secret $\mathbb{C}_{r1} \in \mathbb{Z}^*_{\mathbb{P}_n}$ and also add current timestamp ($\mathbb{TSTP}_{\mathbb{CLOUD}}$) and compute $\mathbb{mCLOUD}$= $H_f(\mathbb{PRID}_{\mathbb{CLOUD}} \parallel \mathbb{TMID}_{\mathbb{CLOUD}} \parallel \mathbb{TPC}_{\mathbb{CLOUD}} \parallel \mathbb{CLOUD}_{\mathbb{prkey}} \parallel \mathbb{TSTP}_{\mathbb{CLOUD}} \parallel \mathbb{C}_{r1})$, and creates session key ($\mathbb{CLOUD}_{\mathbb{SESKEY}}$) using $\mathbb{mCLOUD}$ * $\mathbb{Q}$. Next, $\mathbb{CLOUD}$ creates signatures $\mathbb{CLOUD}_{\mathbb{sign2}}$= $\mathbb{mCLOUD} \oplus H_f(\mathbb{TMID}_{\mathbb{CLOUD}} \parallel \mathbb{TMID}_{\mathbb{CLOUD}} \parallel \mathbb{CLOUD}_{\mathbb{pbkey}} \parallel \mathbb{TSTP}_{\mathbb{CLOUD}})$ * $\mathbb{CLOUD}_{\mathbb{prkey}}$ (mod $\mathbb{Q}$). Next, $\mathbb{CLOUD}$ sends message $\mathbb{CLOUD}_{\mathbb{msg}}$ = ($\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{CLOUD}_{\mathbb{SESKEY}}$, $\mathbb{CLOUD}_{\mathbb{sign2}}$, $\mathbb{TSTP}_{\mathbb{CLOUD}}$, $\mathbb{CLOUD}_{\mathbb{pbkey}}$) to $\mathbb{FOG}$ using secure channel.

STEP-3: $\mathbb{FOG}$ accepts the message and matches session key $\mathbb{CLOUD}_{\mathbb{SESKEY}}$ using $\mathbb{mCLOUD}$ * $\mathbb{CLOUD}_{\mathbb{SESKEY}}$, and also verifies the timestamp $\mathbb{TSTP}^*_{\mathbb{CLOUD}}$ with $|\mathbb{TSTP}^*_{\mathbb{CLOUD}} - \mathbb{TSTP}_{\mathbb{CLOUD}}|$ leq $\delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully, then it verifies the signature $\mathbb{CLOUD}_{\mathbb{sign2}}$ by $\mathbb{CLOUD}_{\mathbb{sign2}}$ * $\mathbb{Q}$ = $H_f(\mathbb{TMID}_{\mathbb{CLOUD}} \parallel \mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{CLOUD}_{\mathbb{pbkey}} \parallel \mathbb{TSTP}_{\mathbb{CLOUD}} \parallel \mathbb{CLOUD}_{\mathbb{SESKEY}})$. If the signature matches successfully, then $\mathbb{FOG}$ creates new timestamp $\mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}}$ and compute $\mathbb{mFOG}_{New}$ = $H_f(\mathbb{CLOUD}_{\mathbb{SESKEY}} \parallel \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}})$ and sends acknowledgement $\mathbb{FOG}_{\mathbb{ACKW}}$= $\{\mathbb{CLOUD}_{\mathbb{SESKEY}}, \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}}\}$ to $\mathbb{CLOUD}$ using secure channel.

STEP-4: $\mathbb{CLOUD}$ accepts the message matches session key $\mathbb{FOG}_{\mathbb{SESKEY}}$ using $H_f(\mathbb{CLOUD}_{\mathbb{SESKEY}} \parallel \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}})$, and also verifies the timestamp $\mathbb{TSTP}^*_{\mathbb{FOG}_{\mathbb{New}}}$ with $|\mathbb{TSTP}^*_{\mathbb{FOG}_{\mathbb{New}}} - \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}}|$ leq $\delta\mathbb{T}$, where $\delta\mathbb{T}$ denotes maximum delay time. If both conditions matches successfully, then both $\mathbb{FOG}$ and $\mathbb{CLOUD}$ shares the secret key $\mathbb{FOG}_{\mathbb{SKEY}}$ = $\mathbb{CLOUD}_{\mathbb{SKEY}}$ for further communication. The detailed authentication process of FOG-to-CLOUD is described in Table VIII.

Table VIII: Fog-to-Cloud Authentication

FOG ($\mathbb{FOG}$)	**Cloud** ($\mathbb{CLOUD}$)
Select secret number ($\mathbb{F}_{r1}$) $\in \mathbb{Z}^*_{\mathbb{P}_n}$ Find $\mathbb{mFOG}$= $H_f(\mathbb{PRID}_{\mathbb{FOG}} \parallel \mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{TPC}_{\mathbb{FOG}} \parallel \mathbb{FOG}_{\mathbb{prkey}} \parallel \mathbb{TSTP}_{\mathbb{FOG}} \parallel \mathbb{F}_{r1})$ Compute signature, $\mathbb{FOG}_{\mathbb{sign1}}$= $\mathbb{mFOG} \oplus H_f(\mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{FOG}_{\mathbb{prkey}} \parallel \mathbb{TSTP}_{\mathbb{FOG}})$ * $\mathbb{FOG}_{\mathbb{prkey}}$ (mod $\mathbb{Q}$) Send Message, $\mathbb{FOG}_{\mathbb{msg}}$ = ($\mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{FOG}_{\mathbb{sign1}} \parallel \mathbb{TSTP}_{\mathbb{FOG}} \parallel \mathbb{FOG}_{\mathbb{pbkey}}$) to $\mathbb{CLOUD}$	
	verify signature $\mathbb{FOG}_{\mathbb{sign1}}$ by $\mathbb{FOG}_{\mathbb{sign1}}$ * $\mathbb{Q}$ = $H_f(\mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{FOG}_{\mathbb{pbkey}} \parallel \mathbb{TSTP}_{\mathbb{FOG}})$ and also check timestamp $\mathbb{TSTP}^*_{\mathbb{FOG}}$ with $|\mathbb{TSTP}^*_{\mathbb{FOG}} - \mathbb{TSTP}_{\mathbb{FOG}}|$ leq $\delta\mathbb{T}$ Creates session key ($\mathbb{CLOUD}_{\mathbb{SESKEY}}$) using $\mathbb{mCLOUD}$ * $\mathbb{Q}$ Create signature $\mathbb{CLOUD}_{\mathbb{sign2}}$= $\mathbb{mCLOUD} \oplus H_f(\mathbb{TMID}_{\mathbb{CLOUD}} \parallel \mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{CLOUD}_{\mathbb{pbkey}} \parallel \mathbb{TSTP}_{\mathbb{CLOUD}})$ * $\mathbb{CLOUD}_{\mathbb{prkey}}$ (mod $\mathbb{Q}$) Send message $\mathbb{CLOUD}_{\mathbb{msg}}$ = ($\mathbb{TMID}_{\mathbb{CLOUD}}$, $\mathbb{CLOUD}_{\mathbb{SESKEY}}$, $\mathbb{CLOUD}_{\mathbb{sign2}}$, $\mathbb{TSTP}_{\mathbb{CLOUD}}$, $\mathbb{CLOUD}_{\mathbb{pbkey}}$) to $\mathbb{FOG}$ using secure channel
checks the session key $\mathbb{CLOUD}_{\mathbb{SESKEY}}$ using $\mathbb{mCLOUD}$ * $\mathbb{CLOUD}_{\mathbb{SESKEY}}$, and also verifies the timestamp $\mathbb{TSTP}_{\mathbb{CLOUD}}$ with $|\mathbb{TSTP}^*_{\mathbb{CLOUD}} - \mathbb{TSTP}_{\mathbb{CLOUD}}|$ leq $\delta\mathbb{T}$, where δ Verify signature $\mathbb{CLOUD}_{\mathbb{sign2}}$ by $\mathbb{CLOUD}_{\mathbb{sign2}}$ * $\mathbb{Q}$ = $H_f(\mathbb{TMID}_{\mathbb{CLOUD}} \parallel \mathbb{TMID}_{\mathbb{FOG}} \parallel \mathbb{CLOUD}_{\mathbb{pbkey}} \parallel \mathbb{TSTP}_{\mathbb{CLOUD}} \parallel \mathbb{CLOUD}_{\mathbb{SESKEY}})$ Create new timestamp $\mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}}$ and compute $\mathbb{mFOG}_{New}$ = $H_f(\mathbb{CLOUD}_{\mathbb{SESKEY}} \parallel \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}})$ Send acknowledgement $\mathbb{FOG}_{\mathbb{ACKW}}$= $\{\mathbb{CLOUD}_{\mathbb{SESKEY}}, \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}}\}$ to $\mathbb{CLOUD}$ using secure channel	
	$\mathbb{CLOUD}$ receives the message and checks the session key $\mathbb{FOG}_{\mathbb{SESKEY}}$ using $H_f(\mathbb{CLOUD}_{\mathbb{SESKEY}} \parallel \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}})$, and also verifies the timestamp $\mathbb{TSTP}^*_{\mathbb{FOG}_{\mathbb{New}}}$ with $|\mathbb{TSTP}^*_{\mathbb{FOG}_{\mathbb{New}}} - \mathbb{TSTP}_{\mathbb{FOG}_{\mathbb{New}}}|$ leq $\delta\mathbb{T}$ If both the conditions are matches, then both $\mathbb{FOG}$ and $\mathbb{CLOUD}$ shares the secret key $\mathbb{FOG}_{\mathbb{SKEY}}$ = $\mathbb{CLOUD}_{\mathbb{SKEY}}$ for further communication.

(iv) Block Creation and Validation Phase

Once the $\mathbb{IOTD}$ is registered by $\mathbb{TP}$, the process of joining blockchain is started. The ePoW consensus is applied to validate and create a block in the blockchain network. The system constructs a transaction-based polynomial to implement conformance proof (ePoW) in the network.

The proposed framework uses a polynomial of transactions to implement proof of signature and identity using smart contract enables ePoW approach. Let V= $\{V_1,V_2,V_3,\ldots,V_m\}$ denotes transactions. Transaction hash is evaluated using $HS_1(V_1)$,$HS_1(V_2)$,...,$HS_1(V_m)$ and produces a polynomial f(z) with order of m, such as f($HS_1(v_u)$)= 0, u$\in$ $\{1,2,3,4,\ldots,m\}$. Further, polynomial is constructed as

$$f(z) = (z - HS_1(V_1))(z - HS_1(V_2)) \ldots (z - HS_1(V_m)). \quad (1)$$

Equation 1 is formulated further as

$f(z) = z^m + c_{m-1}z^{m-1} + \cdots + c_1 z + c_0$

where [1,c_{m-1},c_{m-2},...,c_0] are the coefficient. Next, when $f(z) = 0$ then, Equation 1 is formulated as

$$z^m + c_{m-1}z^{m-1} + \cdots + c_1 z = -c_0. \quad (2)$$

Equation 2 is further divided by -c_0 both side and formulated as

$$\frac{-1}{c_0}z^m + \frac{-c_{m-1}}{c_0}z^{m-1} + \cdots + \frac{-c_1}{c_0}z = 1. \quad (3)$$

Let r_m=$\frac{-1}{c_0}$, r_{m-1}=$\frac{-c_{m-1}}{c_0}$,...,r_1=$\frac{-c1}{-a0}$ produces polynomial.

$$w(z) = r_m z^m + r_{m-1}z^{m-1} + \cdots + r_1 z. \quad (4)$$

It is further summarized as w($HS_1(V_u)$)=1, where $V_u \in V$. Next, R vector is defined as = $\{r_1,r_2,\ldots,r_{m-1},r_m\}$ and other vector hs_j is defined as hs_j= [$HS_1(v_u)$, $(HS_1(v_u))^2$,..., $(HS_1(v_u))^{m-1}$, $(HS_1(v_u))^m$]. Then, the product of both the vector r and hs_u is r.hs_u=1. Here, the "r" vector is denoted as consensus vector and it is verified each time when a new block index gets generated in the network. If more than $\frac{1}{3}$ of the fog peers verifies the transaction (V_u) then, it gets appended as a new valid block index.

The steps used in authenticating data using proposed ePoW are described in Algorithm 1.

The proposed algorithm consists of three separate functions a) creation of block b) ePoW based block mining, and c) adding a new block. The first function generates a block hash using Previous_hash, Block_index, transaction, timestamp, proof, Current_hash, and SHA512. In the ledger, this hash is accessed by subsequent blocks.

As stated earlier (see Section I), compared to actual [18] storage, blockchain is effective in storing hashes. The observational data (sensor collected information) is thus, stored in the distributed hash table (DHT) of the IPFS storage layer with its corresponding addressable hash. This storage ensures security, immutable, and removes similar data (repletion of same data pattern) for storage. Besides, the storage layer maintains version control where each data record provided a unique hash address (content-addressable hash). Algorithm 2 discusses steps of the IPFS storage layer for each original transaction record.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

The process of validation and block creation is shown in Table IX. The steps of the process between $\mathbb{IOTD}$ and $\mathbb{FOG}$ is described below:

Step 1: In the initial step, $\mathbb{IOTD}$ key pair ($\mathbb{IOTD}_{pbkey}$, $\mathbb{IOTD}_{prkey}$), where $\mathbb{IOTD}_{pbkey}$ denotes public key and $\mathbb{IOTD}_{prkey}$ denotes private key of the IoT devices ($\mathbb{IOVN}$). Next, the work of $\mathbb{FOG}$ is initiated.

Step 2: Now, $\mathbb{FOG}$ generates signature ($\mathbb{FOG}_{\mathtt{sign}}$) and send it to the $\mathbb{IOTD}$ for validation.

Step 3: The $\mathbb{IOTD}$ validate signature for verification. If $\mathbb{FOG}_{\mathtt{sign}}$ is valid, $\mathbb{IOTD}$ request to $\mathbb{FOG}_{\mathtt{sign}}$ joining of blockchain network with its $\mathbb{IOTD}_{pbkey}$ and $\mathbb{ID}_{\mathbb{IOTD}}$.

Step 4: Next, $\mathbb{IOTD}$ sends $\mathbb{IOTD}_{\mathtt{sign}}$ to the FOG units peer nodes ($\mathbb{FOG}_{\mathtt{Peer}}$) for validation of public key of $\mathbb{IOTD}_{pbkey}$.

Step 5: The peer nodes ($\mathbb{FOG}_{\mathtt{Peer}}$) validates public key and signature of $\mathbb{IOTD}_{\mathtt{sign}}$ and send back as status of True/ False.

Step 6: If the status assigned with True, new block ($\mathbb{ID}^{\mathbb{BLOCK}}_{\mathbb{IOTD}}$) is created and added to the blockchain associated with $\mathbb{IOTD}_{pbkey}$ and $\mathbb{ID}_{\mathbb{IOTD}}$. Further, actual data is stored into the IPFS based secured distributed storage layer.

(v) Data Generation and Block Updation

This phase discusses data generation process (how $\mathbb{IOTD}$ creates transaction ($\mathbb{ID}^{\mathbb{TX}}_{\mathbb{IOTD}}$) with block updates). The data creation and block update phase are summarized in Table X. The steps of the entire process for data creation and block updation are described below.

Step 1: In initial step, the transactions (TX) is created from the $\mathbb{ID}_{\mathbb{IOTD}_{\mathbb{TX}}}$ is signed ($\mathbb{IOTD}_{\mathtt{sign}}$) using $\mathbb{IOTD}_{prkey}$ of $\mathbb{ID}_{\mathbb{IOTD}}$, once the $\mathbb{ID}^{\mathbb{TX}}_{\mathbb{IOTD}}$ is signed, new transaction ($\mathbb{ID}^{\mathbb{NEWTX}}_{\mathbb{IOTD}}$) is created along with $\mathbb{IOTD}_{\mathtt{sign}}$, $\mathbb{IOTD}_{pbkey}$, $\mathbb{ID}_{\mathbb{IOTD}}$ of $\mathbb{ID}_{\mathbb{IOTD}}$. Next $\mathbb{ID}^{\mathbb{NEWTX}}_{\mathbb{IOTD}}$ is send to the $\mathbb{FOG}_{\mathtt{Peer}}$ for validation and further updation in $\mathbb{IOTD}_{\mathbb{BLOCK}}$.

Step 2: Next, valid record of the $\mathbb{IOTD}_{pbkey}$ and associated $\mathbb{ID}_{\mathbb{IOTD}}$ is checked along with $\mathbb{ID}^{\mathbb{TX}}_{\mathbb{IOTD}}$ and $\mathbb{IOTD}_{\mathtt{sign}}$. If all are valid then, $\mathbb{ID}^{\mathbb{NEWTX}}_{\mathbb{IOTD}}$ is appended in a block $\mathbb{IOTD}_{\mathbb{BLOCK}}$ and disseminated to blockchain network.

Step 3: Finally, the $\mathbb{IOTD}_{\mathbb{BLOCK}}$ is updated and added to the blockchain network.

(vi) Dynamic IoT device Addition

This phase discusses dynamic IoT device addition into the proposed model. The detailed process of dynamic IoT device registration is discussed below. STEP-1 : The $\mathbb{TP}$ selects real identity of IoT device ($\mathbb{ID}^{\mathbb{DYN}}_{\mathbb{IOTD}}$) and picks temporary identity ($\mathbb{TMID}^{\mathbb{DYN}}_{\mathbb{IOTD}}$) , and chooses the random secret ($\mathbb{R}_{\mathtt{s1}}{}^{DYN}$) $\in \mathbb{Z}^*_{\mathbb{P}_n}$, to find the pseudo random identity ($\mathbb{PRID}^{\mathbb{DYN}}_{\mathbb{IOTD}}$) of the $\mathbb{IOTD}^{DYN}$ using $\mathbb{PRID}^{\mathbb{DYN}}_{\mathbb{IOTD}} = H_f(\mathbb{ID}^{\mathbb{DYN}}_{\mathbb{IOTD}} \mid\mid \mathbb{R}_{\mathtt{s1}}{}^{DYN} \mid\mid \mathbb{MKY}_{\mathbb{TP}})$, where $\mathbb{MKY}_{\mathbb{TP}}$ denotes the master key of the trusted authority ($\mathbb{TP}$).

Algorithm 1: Data records Authentication using an enhanced Proof-of-Work (ePoW) algorithm

Input: transaction (T_{xn}),Previous_Hash, Block_Index, timestamp, transactions, proof, Current_hash, SHA512
Result: Creation of Block_Hash (adding the last block in chain)
Initialization:
$Block\_Index = 0$
$Previous\_Hash = 0$
/* Creation of Block and Its Corresponding Hash */
create_block(proof):
if *(T_{xn} is valid)* **then**
  computeHash(Block_Hash)
end
if $(Block\_Index >= 0)$ **then**
  Block_Hash= Digest(Block_Index, Previous_Hash, timestamp, transactions, proof, Current_hash, SHA512)
end
Return Block_Hash
/* Process of Block Mining using Enhanced Proof-of-Work (ePoW) */
block_mining(last_proof):
$proof \leftarrow last\_proof + 1$
while *(((proof + last_proof) &(2^n - 1))==0)* **do**
  $proof \leftarrow proof + 1$
end
Return proof
/* Process of adding Block into Blockchain Network */
add_block(new_block):
/* create the proof for number of transactions (N) records in Datasets */
for *i in 1:N* **do**
  if *(i==1)* **then**
    proof=1 /*Genesis Block Creation*/
    add_block = create_block(proof)
  else
    last_proof = i
    ePoW= block_mining(last_proof)
    add_block = create_block(ePoW)
  end
end

Algorithm 2: Off-chain data storage layer using IPFS

Input: transaction Sensor node (S) information
Result: Creation of content addressable hash
Initialization:
/*Initialization of content addressed hash ($\text{Cnt}_{address}$) and transaction (T_{xn}) to 0 */
$T_{xn} = 0$
$\text{Cnt}_{address} = 0$
for *All Sensor node (Sk)* **do**
  T_{xn} = Retrieve values from the sensor node
  if *(T_{xn} is valid)* **then**
    $\text{Cnt}_{address}$=Compute the content-addressed hash (T_{xn})
    Store the $\text{Cnt}_{address}$ information to IPFS nodes
    IPFS nodes stores these information in distributed hash table (DHT)
    Send the addressable hash into BlockFog and BlockCloud for further access of actual details of the data
  end
end

Table IX: Process of validation and block creation

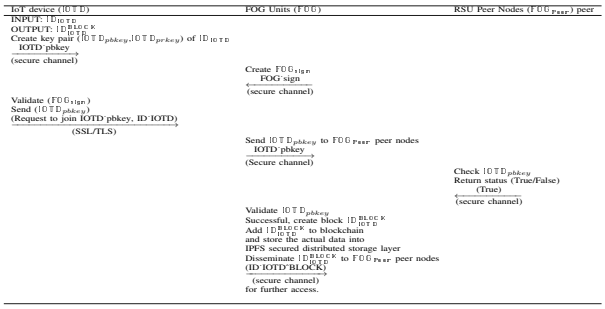
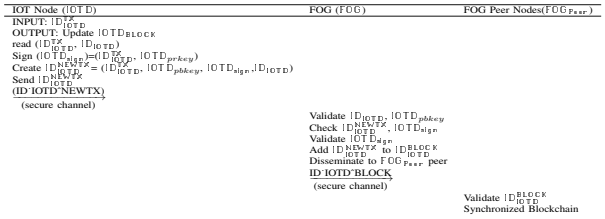


Table X: Process of data creation and updation of Block



This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

STEP-2 : The $\mathbb{TP}$ selects random private key ($\mathbb{IOTD}_{prkey}^{DYN}$) $\in \mathbb{Z}_{\mathbb{P}_n}^*$ and finds the corresponding public key ($\mathbb{IOTD}_{pbkey}^{DYN}$) = $\mathbb{IOTD}_{prkey}^{DYN}$ x $\mathbb{Q}$, and computes the temporary credential ($\mathbb{TPC}_{\mathbb{IOTD}}^{\mathbb{DYN}}$) = $H_f(\mathbb{PRID}_{\mathbb{IOTD}}^{\mathbb{DYN}} \parallel \mathbb{IOTD}_{prkey}^{DYN} \parallel \mathbb{MKY}_{\mathbb{TP}} \parallel \mathbb{TSTP}_{\mathbb{IOTD}}^{\mathbb{DYN}})$, where $\mathbb{TSTP}_{\mathbb{IOTD}}^{\mathbb{DYN}}$ denotes timestamp of $\mathbb{IOTD}^{DYN}$ registration.

STEP-3 : The $\mathbb{TP}$ sets all the required credential for the IoT device ($\mathbb{IOTD}^{DYN}$) with { ($\mathbb{PRID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TMID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TPC}_{\mathbb{IOTD}}^{\mathbb{DYN}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$, ($\mathbb{IOTD}_{prkey}^{DYN}$, $\mathbb{IOTD}_{pbkey}^{DYN}$) }. Next, $\mathbb{TP}$ disseminate the $\mathbb{IOTD}_{pbkey}^{DYN}$ key of respective IoT device ($\mathbb{IOTD}^{DYN}$) for further use. The summary of dynamic IoT device registration is discussed in Table XI.

Table XI: Summary of dynamic IoT Device registration process

Trusted Party ($\mathbb{TP}$)	**Dynamic IoT Device** ($\mathbb{IOTD}^{DYN}$)
$\mathbb{TP}$ selects $\mathbb{ID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TMID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{R}_{s\mathbb{1}}{}^{DYN} \in \mathbb{Z}_{\mathbb{P}_n}^*$ Find $\mathbb{PRID}_{\mathbb{IOTD}}^{\mathbb{DYN}} = H_f(\mathbb{ID}_{\mathbb{IOTD}}^{\mathbb{DYN}} \parallel \mathbb{R}_{s\mathbb{1}}{}^{DYN} \parallel \mathbb{MKY}_{\mathbb{TP}})$ $\mathbb{TP}$ selects $\mathbb{IOTD}_{prkey}^{DYN} \in \mathbb{Z}_{\mathbb{P}_n}^*$ Compute $\mathbb{IOTD}_{pbkey}^{DYN} = \mathbb{IOTD}_{prkey}^{DYN}$ x $\mathbb{Q}$ Finds ($\mathbb{TPC}_{\mathbb{IOTD}}^{\mathbb{DYN}}$) = $H_f(\mathbb{PRID}_{\mathbb{IOTD}}^{\mathbb{DYN}} \parallel \mathbb{IOTD}_{prkey}^{DYN} \parallel \mathbb{MKY}_{\mathbb{TP}} \parallel \mathbb{TSTP}_{\mathbb{IOTD}}^{\mathbb{DYN}})$ Initialize $\mathbb{IOTD}^{DYN}$ with {($\mathbb{PRID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TMID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TPC}_{\mathbb{IOTD}}^{\mathbb{DYN}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$, ($\mathbb{IOTD}_{prkey}^{DYN}$, $\mathbb{IOTD}_{pbkey}^{DYN}$) }	Stores {($\mathbb{PRID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TMID}_{\mathbb{IOTD}}^{\mathbb{DYN}}$, $\mathbb{TPC}_{\mathbb{IOTD}}^{\mathbb{DYN}}$), $H_f(.)$, $\mathbb{E}_{\mathbb{P}_n}$(r,s), $\mathbb{Q}$, ($\mathbb{IOTD}_{prkey}^{DYN}$, $\mathbb{IOTD}_{pbkey}^{DYN}$) } inside memory for further use.

(vii) Encryption and Decryption of IoT Device ($\mathbb{IOTD}$) Generated Data: Once the IoT device registered by $\mathbb{TA}$ successfully, then it obtains public key $\mathbb{IOTD}_{pbkey}$,private key $\mathbb{IOTD}_{prkey}$. By successful authentication of an IoT device $\mathbb{IOTD}$, secret key $\mathbb{IOTD}_{SKEY}$ is created on the infinite field of $\mathbb{Z}_{\mathbb{P}_n}^*$ with random point $\mathbb{R}_s$ on elliptic curve. The $\mathbb{IOTD}_{SKEY}$ is generated with the summation of $\mathbb{IOTD}_{pbkey}$, $\mathbb{IOTD}_{prkey}$, and $\mathbb{R}_s$ shown in Equation 5.

$$\mathbb{IOTD}_{SKEY} = \sum(\mathbb{IOTD}_{pbkey}, \mathbb{IOTD}_{prkey}, \mathbb{R}_s) \tag{5}$$

Here $\mathbb{IOTD}_{SKEY}$ denotes the secret key. After successful creation of the secret key, the data obtained from the $\mathbb{IOTD}$ is encrypted. The encrypted infromation contains the two different cypertext that is mathematically written in Equation 6 and Equation 7, respectively.

$$\mathbb{CP1} = (\mathbb{R}_{s\mathbb{1}} * \mathbb{R}_s) + \mathbb{IOTD}_{SKEY}. \tag{6}$$

$$\mathbb{CP2} = \mathbb{IOTD}_{msg} + (\mathbb{R}_{s\mathbb{1}} * \mathbb{IOTD}_{pbkey}) + \mathbb{IOTD}_{SKEY}. \tag{7}$$

Here $\mathbb{CP1}$ and$\mathbb{CP2}$ denotes the cyper text, $\mathbb{R}_{s\mathbb{1}}$ denotes random $\in \mathbb{Z}_{\mathbb{P}_n}^*$, and $\mathbb{IOTD}_{msg}$ is the actual message generated by the IoT device. The original message is decrypted by the following procedure shown in Equation 8.

$$\mathbb{IOTD}_{msg} = ((\mathbb{CP2} - \mathbb{IOTD}_{prkey}) * \mathbb{CP1}) - \mathbb{IOTD}_{SKEY}. \tag{8}$$

$ii)$ ***Privacy using Principal Component Analysis:*** Once the data gets authenticated, a block is generated using the first level of the privacy-preservation process and is successfully disseminated into the blockchain network. Then the second level of privacy is successfully imposed on raw data collected from IoT devices. The second level of privacy includes three key steps; $i)$ feature mapping, $ii)$ feature selection, and $iii)$ feature transformation. First, in feature mapping, the categorical variable values are converted into a numeric type. The reason for this conversion is that the Pearson correlation coefficient (PCC) and PCA, mechanism works efficiently with numeric values, compared to categorical values [40]. Second, feature selection is a technique of selecting an appropriate and relevant feature from a given set of features and then dismissing the irrelevant attributes. The primary objective of the feature selection methodology is to select features that better reflect the given problem with minimal performance degradation. PPSF framework uses PCC-based simplest statistical technique for feature selection. This approach measures the similarity between the given two features, such that the most significant lowest N ranked features are selected. We can summarize this technique [40], as for the given two random feature k_1 and k_2, their PCC is computed using Equation 9.

$$PCC(k_1, k_2) = \frac{\sum_{i=1}^{m}(x_i - \overline{k_1})(y_i - \overline{k_2})}{\sqrt{\sum_{i=1}^{m}(x_i - \overline{k_1})^2}\sqrt{\sum_{i=1}^{m}(y_i - \overline{k_2})^2}}. \tag{9}$$

In above Equation 9, x_i and y_i are the data points for the given features. Absolute means of k_1 and k_2 are denoted by $\overline{k_1} = \left|\frac{1}{m}\sum_{i=1}^{m} x_i\right|$, $\overline{k_2} = \left|\frac{1}{m}\sum_{i=1}^{m} y_i\right|$ respectively. The result of Equation 9 is between $[-1, +1]$. The PCC-based feature selection technique tries to measure the linear dependence of the two given features, such that for two dependent features, the PCC value will be ± 1, however, in case of independent features, there PCC value will be 0. As a function Equation 9, is used to achieve an optimal feature set, and further fed as an input to PCA for encoding.

Third, the PCA technique is used to transform the optimized attribute collected into a new dimension, such that private knowledge stays concealed in the raw data. PCA is a statistical method that uses an orthogonal transformation to transform the given features into a set of linearly uncorrelated features without losing much information from the data. A new encoded transformation called principal components is the received output. In PPSF, from the highest potential variance to the lowest, we have ordered the obtained components. Thus, we assume that the first component covers the highest variation relative to other components in the resulting data collection. The following is a general outline of the PCA transformation used in PPSF [41]. Suppose that $DT(a) = \{(p_i, q_i)\}_{i=1}^{M}$ for $a = 1, 2, \ldots, k$ is a dataset collected with zero mean values and attributes, where p_i is the data sample and q_i is the target class labels. By using Equation 10, we can calculate the $DT(a)$ co-variance matrix.

$$Q = \frac{1}{k-1}\sum_{a=1}^{k}[DT(a)D(a)^T]. \tag{10}$$

After co-variance, we compute the linear transformation from $DT(a)$ to $L(a)$ using Equation 11.

$$L(a) = N^T DT(a), \tag{11}$$

where N is an orthogonal matrix whose i^{th} covariance matrix column Q is equal to the i^{th} covariance matrix. Using Equation 12, we can fix the eigenvalue problem;

$$\lambda_i s_i = Q s_i, \tag{12}$$

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

where λ_i denotes eigenvalue of Q (consider $\lambda_1 >, \lambda_2 > \ldots \lambda_k$). The related eigenvector is s_i. The principal components are now calculated from Equation 11 using Equation 13;

$$L_i(a) = s_i^T DT(a), i = 1, \ldots k \tag{13}$$

$L_i(a)$ represents i^{th} principal component. In addition, the k eigenvectors are sorted in decreasing order by using the λ_i eigenvalues to perform feature transformation. We will use Equation 14 to measure the projection of the new sample $DT(a)$ into principal space.

$$\hat{DT}(a) = \sum_{i=1}^{n} c_i^T DT(a) c_i, \tag{14}$$

where $C = \{c_i : c_i = s_i, i = 1, 2 \ldots n\}$. In addition, using Equation 15, we can compute the projection error err_a of $DT(a)$ by calculating distance dis_s between $DT(a)$ and $\hat{DT}(a)$,

$$err_a = dis_s(DT(a), \hat{DT}(a)). \tag{15}$$

PPSF framework uses the above equations as a function to transform the obtained optimized dataset into a new encoded format that prevents inference attacks in the proposed model.

B. Gradient Boosting Anomaly detector

Once the PCA transformation is performed on the optimized dataset, on the resultant principal component, a Gradient Boosting Anomaly Detector (GBAD) based LightGBM technique is applied as a utility system. LightGBM uses the prediction results of multiple decision trees (DT) to make the final prediction. The reason of using LightGBM in the proposed PPSF framework is that, first, it integrates multiple "weak" learner into a "strong" one and acquiring "weak" learners are easy. Second, compared to the single learner, multiple learners have better generalization ability [27].

Algorithm 3 illustrates the step-by-step working of LightGBM. As from Equation 14 of two-level privacy-preservation module, we obtained a transformed dataset $\hat{DT}(a)$=$\{(p_i, q_i)\}_{i=1}^{M}$, where p_i are the data samples and q_i are the target class labels. $\widehat{q}_i^{(t)}$ is the prediction result for i^{th} instance at the t^{th} iteration. $f_t p(i)$ denotes the learned function of t^{th} DT. $L_{(t)}$ represents the loss function that measures the difference between predicted $\widehat{q}_i^{(t)}$ and actual (target) $q_i^{(t)}$ label. We can further add a regularization term that can help in penalizing the complexity of overall model. The termination condition for the model is when the training process finishes m^{th} iteration.

C. Proposed PPSF deployment architecture in smart city

The deployment architecture for the proposed PPSF framework is presented in this section. Fig. (2) indicates the working of the proposed model. On the fog and cloud side, the PPSF framework is deployed to overcome the shortcomings of existing deployment techniques focused on fog or fog-cloud.

The systematic deployment architecture at fog and cloud is divided into two main components; blockchain-IPFS integrated fog and blockchain-IPFS integrated cloud discussed below:

Algorithm 3: LightGBM model training for smart city

1 **Input:** Transformed Dataset $\hat{DT}(a)$=$\{(p_i, q_i)\}_{i=1}^{M}$
2 **Output:** LightGBM final model $\widehat{q}_i^{(t)}$
3 **Initialization:**
4 /* Initialization of first tree as a constant */
5 $\widehat{q}_i^{(0)} = f_0 = 0$
6 /* Next tree is trained for minimizing loss function */
7 $f_t p(i) = arg \min_{f_t} L_{(t)} = arg \min_{f_t} L(q_i \widehat{q}_i^{(t-1)} + f_t p(i))$
8 Take the next model in an additive procedure:
9 $\widehat{q}_i^{(t)} = \widehat{q}_i^{(t-1)} + f_t p(i)$
10 Repeat Step 7 and Step 9 until the model reaches the termination condition.
11 Obtain and return the final model:
12 $\widehat{q}_i^{(t)} = \sum_{t=0}^{M-1} f_t p(i)$

Table XII: Selected set of features from ToN-IoT and BoT-IoT by PCC technique

Dataset	Features selected	Feature description
ToN-IoT	19	F16, F17, F7, F8, F9, F1, F11, F19, F2, F4, F6, F10, F15, F3, F12, F5, F13, F14, F18.
BoT-IoT	10	F6, F11, F1, F5, F10, F2, F8, F4, F7, F9.

i) ***Blockchain-IPFS integrated Fog:*** The IoT devices at the device layer are responsible to generate transactions. These resource-limited IoT devices, however, offload their task to nearby fog nodes for computational-intensive activities. Once the generated transaction is forwarded using the gateway/router to the nearest fog node, the actual data of the transactions is stored using Algorithm 2 in the IPFS storage layer. Then, the addressable hash is returned that is used by ePoW based on blockchain and smart contracts, to create a proof, and to construct a hash of a block using the Algorithm 1. In addition, the produced hash will be accessed by the participating IoT nodes for validation and then the block will be added to the chain. Data poisoning attacks i.e., FDIA are prevented by this step. Next, the actual IoT data from IPFS is sent to the PCA-based transformation technique that transforms the actual data into a new encoded format. The above strategy immensely eliminates data inference attacks [4]. The integration of blockchain-IPFS with fog makes the network immutable, auditable, scalable, and verifiable [11], [42]. Finally, GBAD based LightGBM (Algorithm 3) is used to detect an attack and normal observations present in the data. In the case of normal transactions, processing/computation is performed and fog-processed data is sent back to participating IoT nodes. In addition, the security administrator receives an alert when an anomaly is identified to take immediate and temporary action. Finally, the security information of anomalous traffic is sent to the cloud for further processing.

The GBAD based LightGBM as SaaS is installed at each network node, unlike conventional IDSs, and each instance is linked to the IPFS storage layer to capture and record flows in the network. In a particular period, this approach gathers the usable values of network traffic (i.e., by using packet sniffing tool such as Wireshark), which makes it much simpler for each network node (i.e., routers, switches and gateways) to transfer

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

Table XIII: Results of class wise prediction (%) obtained through GBAD using ToN-IoT dataset.

Dataset	Parameters	Backdoor	DDoS	DoS	Injection	MITM	Normal	Password	Ransomware	Scanning	XSS
With Original	PR	100.00	93.00	94.00	94.00	89.00	100.00	97.00	96.00	98.00	91.00
	DR	100.00	91.00	99.00	89.00	78.00	100.00	97.00	98.00	96.00	93.00
	F1	100.00	92.00	96.00	92.00	83.00	100.00	97.00	97.00	97.00	92.00
	FAR	0.00001	0.00294	0.00276	0.00248	0.00017	0.00143	0.00116	0.00202	0.00092	0.00402
With Transformed	PR	100.00	93.00	95.00	94.00	89.00	100.00	97.00	96.00	98.00	92.00
	DR	100.00	91.00	99.00	90.00	80.00	100.00	97.00	98.00	97.00	93.00
	F1	100.00	92.00	97.00	92.00	84.00	100.00	97.00	97.00	97.00	92.00
	FAR	0.00002	0.00327	0.00225	0.00260	0.00021	0.00136	0.00129	0.00183	0.00102	0.00383

Table XIV: Results of class wise prediction (%) obtained through GBAD using BoT-IoT dataset

Dataset	Parameters	DDoS	DoS	Normal	Reconnaissance	Theft
With Original	PR	100.00	100.00	0.00	99.00	93.00
	DR	100.00	100.00	0.00	80.00	93.00
	F1	100.00	100.00	0.00	89.00	93.00
	FAR	0.000031	0.000017	0.004935	0.000128	0.000001
With Transformed	PR	100.00	100.00	90.00	100.00	100.00
	DR	100.00	100.00	93.00	100.00	93.00
	F1	100.00	100.00	92.00	100.00	96.00
	FAR	0.000014	0.00	0.000014	0.000006	0.00

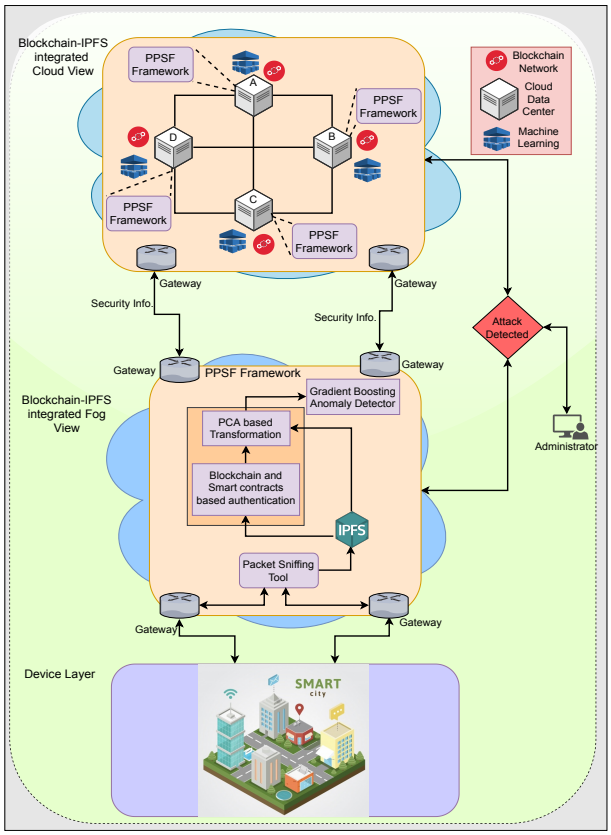


Figure 2: Proposed PPSF deployment architecture in smart city

the processed data to the GBAD based LightGBM module. Additionally, the network data exchanges between the cloud and fog sides are managed and reviewed on the devices and applications of the fog, where network operations of nodes can be tracked.

ii) ***Blockchain-IPFS integrated Cloud:*** On the cloud side, we have different cloud vendors and data centres. These data centres are denoted as $A, B, C,$ and D and are responsible to provide services (such as processing, computation etc.) to customer entities. The PPSF framework is incorporated with a traditional cloud network to form a blockchain-IPFS integrated cloud. This strategy enhances the cloud network and can address the challenges of verifiability, immutability, auditability, and can improve data privacy and further prevents cloud systems from inference and data poisoning attacks [7], [8]. By deploying PPSF at multiple cloud data centres, the incoming traffic is captured, analyzed, transformed and are correlated to differentiate normal traffic from abnormal ones. Additionally, PPSF at various data centres communicates and coordinates with each other to prevent sophisticated intrusions such as Ransomware, MITM, DoS and DDoS. Thus, for normal traffic, required computation is performed and cloud-processed data is sent back to requesting IoT nodes. Moreover, the information is made available to the security administrator when the attack type is identified, to implement complementary and more accurate prevention mechanisms.

IV. Performance Analysis

The PPSF framework is built on Tyrone PC operated by Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz (2 processors), with 128 GB RAM and 2 TB hard disks. In the Scikit-learn library, machine algorithms have been applied. Blockchain technology has been implemented using the programming language of Ethereum and Solidity (version 0.6.50). IPFS has been installed (version 0.4.19). Two intrusion detection datasets namely ToN-IoT [38] and BoT-IoT [37] containing various updated attack observations discovered in IoT networks including Ransomware, DoS, DDoS, Injection and MITM were used for experimentation. ToN-IoT has 1498334 attack and 79053 normal instances. Similarly, BoT-IoT has a 73360900 attack and 9543 normal instances. The efficiency of two-level privacy is evaluated by training and testing the GBAD-based Light GBM model on the original and transformed dataset. As explained in [6], we use AcCuracy (AC), PRecision (PR), Detection Rate/ReCall (DR/RC), F1 and False Alarm Rate (FAR) to evaluate the proposed privacy-preserving approach GBAD. We compare the GBAD-based Light GBM model against state-of-the-art privacy-preserving techniques.

A. The two-level Privacy-Preserving Analysis

We have analyzed the performance of first-level privacy in terms of time taken to upload the actual raw data into the IPFS storage layer, time taken during block mining using ePoW, time for block creation, block access time, and gas price consumption by varying transactions (Tx) and IoT nodes. In Fig. (3) the horizontal axis denotes the number of IoT nodes and the vertical axis represents execution time for different

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

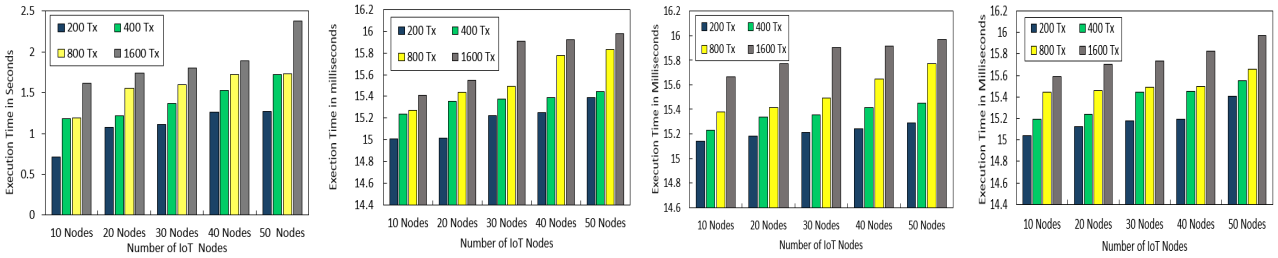
Table XV: Comparison of DR (%) with other ML approaches based on ToN-IoT dataset

Methods	Backdoor	DDoS	DoS	Injection	MITM	Normal	Password	Ransomware	Scanning	XSS
RF	99.98	90.40	91.97	93.53	0.00	100.00	97.81	99.40	95.74	85.47
DT	100.00	100.00	100.00	0.00	0.00	100.00	100.00	100.00	100.00	100.00
NB	99.22	26.80	91.70	92.96	95.11	100.00	75.32	79.98	96.91	19.02
GBAD (with Original)	100.00	91.00	99.00	89.00	78.00	100.00	97.00	98.00	96.00	93.00
GBAD (with Transformed)	100.00	91.00	99.00	90.00	80.00	100.00	97.00	98.00	97.00	93.00

Table XVI: Comparison of DR (%) with other ML approaches based on BoT-IoT dataset

Detection Method	Normal	DDoS	DoS	Reconnaissance	Theft
RF	14.95	99.96	100.00	100.00	0.00
DT	0.00	100.00	100.00	80.06	100.00
NB	74.76	97.76	99.97	81.44	92.85
GBAD (with Original)	100.00	100.00	0.00	80.00	93.00
GBAD (with Transformed)	100.00	100.00	93.00	100.00	93.00

Tx performed by IoT nodes. Fig. (3a) illustrates the execution time to upload raw data into the IPFS storage layer concerning the number of Tx increase. We conclude that upload time increases with an increase in the number of transactions. Fig. (3b) illustrates the time taken by varying IoT nodes concerning a different number of Tx for block mining using proposed ePoW. Fig. (3c) shows a variation of computation time during block creation concerning an increase in the number of Tx

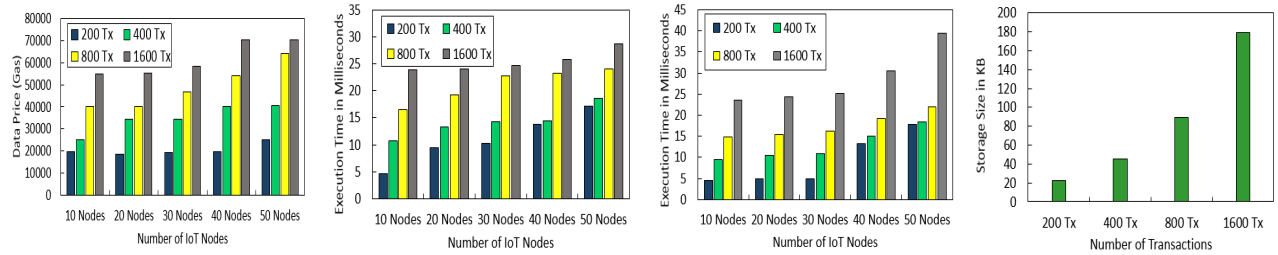


(a) Execution Time Analysis for Actual Data Upload into IPFS Storage Layer

(b) Execution Time Analysis of ePoW Mining for varying sizes of Transaction (Tx)

(c) Execution Time Analysis of block creation for varying sizes of Transaction (Tx)

(d) Execution Time Analysis of block access time for varying sizes of Transaction (Tx)



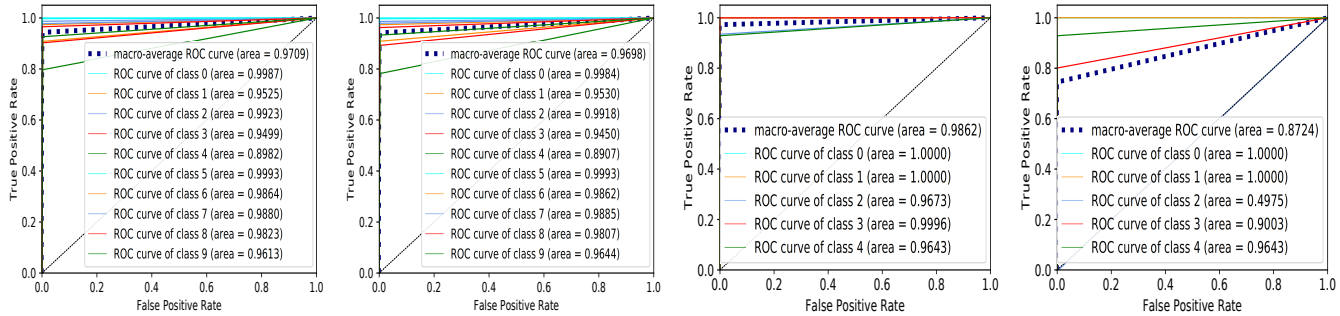
(e) Analysis of Gas price consumption for varying size of Transactions (Tx) on Ethereum blockchain network

(f) Execution Time Analysis of contract deployment for varying size of Transactions (Tx)

(g) Execution Time Analysis of Non-Repudiation for varying sizes of Transaction (Tx)

(h) Analysis of offchain Storage consumption (in KB) for varying size of Transactions (Tx) on IPFS

Figure 3: Results obtained from the proposed privacy-preserving technique based on blockchain and smart contracts



(a) ROC Curve with Transformed ToN-IoT dataset

(b) ROC Curve with Original ToN-IoT dataset

(c) ROC Curve with Transformed BoT-IoT dataset

(d) ROC Curve with Original BoT-IoT dataset

Figure 4: ROC curves obtained from the proposed GBAD approach using both IoT datasets

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TNSE.2021.3089435, IEEE Transactions on Network Science and Engineering

performed by varying IoT nodes. Fig. (3d) shows block access time by participating in IoT nodes for a varying number of Tx. Initially, time increases but it stabilizes after 30 IoT nodes. Fig. (3e) illustrates the change of price trend in the Ethereum network for several Tx added to the network. It can be seen with an increase in the number of Tx, the gas price in the network increases accordingly. Fig. (3f) shows the deployment time of smart contracts taken by the PPSF framework for a set of shared transactions by varying IoT nodes. We conclude that the time to deploy smart contract increases with an increase in IoT nodes. Fig. (3g) states the transaction signing time of varying IoT nodes and transactions. This process ensures non-repudiation in the PPSF framework. It is worth noting that, the transaction signing-time of both 40 and 50 IoT nodes are almost similar for 200 Tx and 400 Tx, respectively. It can be seen that signing time increases with varying transactions. Fig. (3h) illustrates the actual off-chain storage size in KB for varying transactions into the PPSF framework.

This operation is performed to enhance the scalability of the PPSF framework. The actual storage is evaluated in KB, and we conclude that the storage size increases with an increase in transactions.

The transformation based on the PCA technique was used in the second level of privacy protection. The GBAD uses principal components covering maximum cumulative variance from ToN-IoT and BoT-IoT datasets i.e., 9 and 4, respectively.

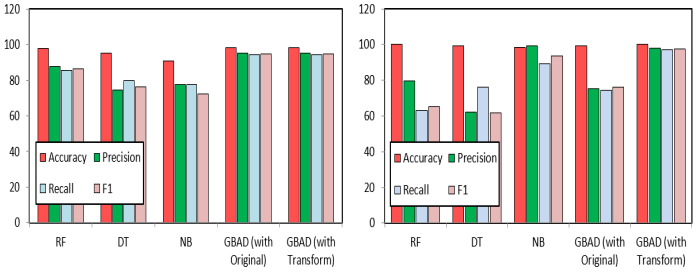
B. Intrusion Detection Analysis

As a utility system of the GBAD, the efficiency of the two-level privacy protection technique is evaluated. The model parameters was configured as, $learning\_rate = 0.1$, $max\_depth = 6$, $subsample = 0.2$, $colsample\_bytree = 0.2$, $reg\_alpha = 0.3$, $reg\_lambda = 0.8$, and $num\_leaves = 5$ for BoT-IoT, and $num\_leaves = 10$ for ToN-IoT datsets respectively to overcome over-fitting and under-fitting. A multi-class classification is performed and evaluated using class wise prediction scores, ROC curves and overall AC, PR, DR and F1 score. The features present in both datasets are preprocessed using steps mentioned in Subsection III-A. Table XII illustrates the feature list selected using PCC in designing GBAD for smart city.

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) for different threshold values. The ROC curve is further evaluated by the area under the curve (AUC) by measuring the area under the curve. An ideal models' AUC will hit 1, while the random classification AUC will result in 0.5. The higher value of the AUC means that the model differentiates the output classes highly efficiently.

Fig. (4a) and Fig. (4b) illustrates the AUC value for each class present in ToN-IoT dataset.

The GBAD using transformed dataset shows AUC values between 0.89 to 0.99 for normal and attack vectors, and more precisely macro-average AUC is comparatively high for transform dataset i.e., 0.97 compared to 0.96 with the original dataset. Similarly, for BoT-IoT dataset, Fig. (4c) and Fig. (4d) shows AUC values for different class vectors. It can be seen that AUC values for normal and attack vectors are high for the transformed dataset. The macro-average AUC for the transformed dataset is 0.98 and for the original is 0.87.



(a) Overall score using ToN-IoT (b) Overall score using BoT-IoT

Figure 5: Comparison of DR AC, PR, RC / DR and F1 score (%) with existing ML approaches on both datasets

Further analysis is conducted based on class-wise prediction scores listed in Table XIII and Table XIV for ToN-IoT and BoT-IoT respectively. We have compared the performance of GBAD with the original and transformed dataset. It can be seen that GBAD with transformed datasets has obtained values between 91% to 100% for PR, DR, and F1. On the other hand, GBAD has reduced FAR for each class close to 0%.

C. Comparisons and discussions

The GBAD-based LightGBM model is compared with three well known ML techniques i.e., random forest (RF) [33], [34], [36]; decision tree (DT) [29], [33], [34], [36]; and Naive Bayes (NB) [29], [33] in terms of overall accuracy (AC), precision (PR), recall (RC/DR), F1 score, and DR under multiclass classification based on two datasets i.e., ToN-IoT and BoT-IoT. Fig. (5) shows the overall performance comparison of the proposed GBAD with other ML techniques. It can be noticed from Figs. (5a) and (5b) that GBAD using ToN-IoT dataset with transformed dataset has achieved significantly better results (i.e., AC of 98.38%, PR of 95.32%, DR of 94.35%, the F1 score is 94.80%). Similarly, using a transformed dataset of BoT-IoT, GBAD has obtained significantly better results (i.e., 99.99% AC, 98.01% of PR, 97.24% DR, and 97.59% F1 score). This is because original data has been encoded by the PCA technique in a new format, and in the obtained principal components variance is high. As a consequence, in both datasets, the separability of malicious and normal data points has become linearly separable. Table XV and XVI shows multi-class DR comparison with existing techniques. The proposed model can detect different attack types from both datasets in an average of $91\% - 100\%$. We can see that for most of the attack vectors including normal vectors, GBAD has achieved comparatively higher values. It can be noticed that the proposed GBAD outperforms other models after applying the proposed two-level privacy preservation technique (transformed dataset).

D. Engineering Applications

The proposed framework using blockchain and machine learning techniques can be used to provide security and

privacy in IoT and its application networks, organizational and social areas where security and privacy are of prime importance. Additionally, the study can also encourage data science researchers to use their work to suggest more complex solutions to the current research issue.

V. Conclusion and Future Work

In this paper, we presented PPSF, an intelligent blockchain framework that integrates blockchain with machine learning techniques to protect privacy and security in IoT-driven smart cities. In privacy preservation, an ePoW technique for verifying data integrity based on blockchain and smart contracts and a PCA-based transformation technique for transforming data into a new format were designed. Compared with recent works, the two-level privacy-preservation approach achieves improved efficiency, since it is immune to inference and data poisoning attacks. A Gradient Boosting Anomaly Detector-based LightGBM was then designed and evaluated on transformed (i.e., after applying privacy-preservation technique) and original (i.e., before applying privacy-preservation technique) ToN-IoT and BoT-IoT based IoT network datasets. We also design an IoT-fog-cloud integrated architecture with blockchain-IPFS to deploy the proposed framework. The findings revealed that the proposed PPSF framework achieves better performance compared with peer privacy-preserving intrusion detection techniques. Future extensions of this work will include designing a prototype to assess the efficiency of the proposed framework.

References

[1] Y. Guo, T. Ji, Q. Wang, L. Yu, G. Min, and P. Li, "Unsupervised anomaly detection in iot systems for smart cities," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 2231–2242, 2020.

[2] E. Ismagilova, L. Hughes, N. P. Rana, and Y. K. Dwivedi, "Security, privacy and risks within smart cities: Literature review and development of a smart city interaction framework," *Information Systems Frontiers*, pp. 1–22, 2020.

[3] H. Dai, Z. Zheng, and Y. Zhang, "Blockchain for internet of things: A survey," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8076–8094, 2019.

[4] M. Keshk, B. Turnbull, N. Moustafa, D. Vatsalan, and K. R. Choo, "A privacy-preserving-framework-based blockchain and deep learning for protecting smart power networks," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 8, pp. 5110–5118, 2020.

[5] K. A. da Costa, J. P. Papa, C. O. Lisboa, R. Munoz, and V. H. C. de Albuquerque, "Internet of things: A survey on machine learning-based intrusion detection approaches," *Computer Networks*, vol. 151, pp. 147–157, 2019.

[6] P. Kumar, G. P. Gupta, and R. Tripathi, "A distributed ensemble design based intrusion detection system using fog computing to protect the internet of things networks," *Journal of Ambient Intelligence and Humanized Computing*, pp. 1–18, 2020.

[7] R. A. Memon, J. Li, J. Ahmed, M. I. Nazeer, M. Ismail, and K. Ali, "Cloud-based vs. blockchain-based iot: a comparative survey and way forward." *Frontiers Inf. Technol. Electron. Eng.*, vol. 21, no. 4, pp. 563–586, 2020.

[8] K. Gai, J. Guo, L. Zhu, and S. Yu, "Blockchain meets cloud computing: A survey," *IEEE Communications Surveys & Tutorials*, 2020.

[9] A. A. Alli and M. M. Alam, "The fog cloud of things: A survey on concepts, architecture, standards, tools, and applications," *Internet of Things*, vol. 9, p. 100177, 2020.

[10] X. Ding, J. Guo, D. Li, and W. Wu, "An incentive mechanism for building a secure blockchain-based internet of things," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2020.

[11] H. Baniata and A. Kertesz, "A survey on blockchain-fog integration approaches," *IEEE Access*, vol. 8, pp. 102 657–102 668, 2020.

[12] A. Garba, Z. Chen, Z. Guan, and G. Srivastava, "Lightledger: A novel blockchain-based domain certificate authentication and validation scheme," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2021.

[13] Z. Li, S. Gao, Z. Peng, B. Xiao, S. Guo, and Y. Yang, "B-dns: A secure and efficient dns based on the blockchain technology," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2021.

[14] J. Du, W. Cheng, G. Lu, H. Cao, X. Chu, Z. Zhang, and J. Wang, "Resource pricing and allocation in mec enabled blockchain systems: An a3c deep reinforcement learning approach," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2021.

[15] H. Yu, Q. Hu, Z. Yang, and H. Liu, "Efficient continuous big data integrity checking for decentralized storage," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2021.

[16] R. Kumar, P. Kumar, R. Tripathi, G. P. Gupta, T. R. Gadekallu, and G. Srivastava, "Sp2f: A secured privacy-preserving framework for smart agricultural unmanned aerial vehicles," *Computer Networks*, vol. 187, p. 107819, 2021.

[17] B. Bhushan, A. Khamparia, K. M. Sagayam, S. K. Sharma, M. A. Ahad, and N. C. Debnath, "Blockchain for smart cities: A review of architectures, integration trends and future research directions," *Sustainable Cities and Society*, p. 102360, 2020.

[18] M. A. Ferrag, M. Derdour, M. Mukherjee, A. Derhab, L. Maglaras, and H. Janicke, "Blockchain technologies for the internet of things: Research issues and challenges," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 2188–2204, 2019.

[19] P. Kumar, G. P. Gupta, and R. Tripathi, "Tp2sf: A trustworthy privacy-preserving secured framework for sustainable smart cities by leveraging blockchain and machine learning," *Journal of Systems Architecture*, p. 101954, 2020.

[20] N. Nizamuddin, K. Salah, M. A. Azad, J. Arshad, and M. Rehman, "Decentralized document version control using ethereum blockchain and ipfs," *Computers & Electrical Engineering*, vol. 76, pp. 183–197, 2019.

[21] M. Al-Hawawreh, N. Moustafa, S. Garg, and M. S. Hossain, "Deep learning-enabled threat intelligence scheme in the internet of things networks," *IEEE Transactions on Network Science and Engineering*, pp. 1–1, 2020.

[22] P. C. M. Arachchige, P. Bertok, I. Khalil, D. Liu, S. Camtepe, and M. Atiquzzaman, "A trustworthy privacy preserving framework for machine learning in industrial iot systems," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 9, pp. 6092–6102, 2020.

[23] O. Alkadi, N. Moustafa, B. Turnbull, and K. R. Choo, "A deep blockchain framework-enabled collaborative intrusion detection for protecting iot and cloud networks," *IEEE Internet of Things Journal*, pp. 1–1, 2020.

[24] S. Zhang and J.-H. Lee, "Analysis of the main consensus protocols of blockchain," *ICT Express*, vol. 6, no. 2, pp. 93–97, 2020.

[25] A. Belhadi, Y. Djenouri, G. Srivastava, and J. C.-W. Lin, "Ss-its: secure scalable intelligent transportation systems," *The Journal of Supercomputing*, pp. 1–17, 2021.

[26] J. M.-T. Wu, G. Srivastava, A. Jolfaei, P. Fournier-Viger, and J. C.-W. Lin, "Hiding sensitive information in ehealth datasets," *Future Generation Computer Systems*, vol. 117, pp. 169–180, 2021.

[27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in neural information processing systems*, 2017, pp. 3146–3154.

[28] P. K. Sharma and J. H. Park, "Blockchain based hybrid network architecture for the smart city," *Future Generation Computer Systems*, vol. 86, pp. 650–655, 2018.

[29] M. Keshk, N. Moustafa, E. Sitnikova, and B. Turnbull, "Privacy-preserving big data analytics for cyber-physical systems," *Wireless Networks*, pp. 1–9, 2018.

[30] S. Rathore, B. W. Kwon, and J. H. Park, "Blockseciotnet: Blockchain-based decentralized security architecture for iot network," *Journal of Network and Computer Applications*, vol. 143, pp. 167–177, 2019.

[31] C. Liang, B. Shanmugam, S. Azam, M. Jonkman, F. De Boer, and G. Narayansamy, "Intrusion detection system for internet of things based on a machine learning approach," in *2019 International Conference on Vision Towards Emerging Trends in Communication and Networking (ViTECoN)*. IEEE, 2019, pp. 1–6.

[32] M. Keshk, E. Sitnikova, N. Moustafa, J. Hu, and I. Khalil, "An integrated framework for privacy-preserving based anomaly detection for cyber-physical systems," *IEEE Transactions on Sustainable Computing*, pp. 1–1, 2019.

[33] M. Shafiq, Z. Tian, A. K. Bashir, X. Du, and M. Guizani, "Iot malicious traffic identification using wrapper-based feature selection mechanisms," *Computers & Security*, p. 101863, 2020.
[34] M. Shafiq, Z. Tian, A. K. Bashir, X. Du, and M. Guizani, "Corrauc: a malicious bot-iot traffic detection method in iot network using machine learning techniques," *IEEE Internet of Things Journal*, pp. 1–1, 2020.
[35] M. Hasan, M. M. Islam, M. I. I. Zarif, and M. Hashem, "Attack and anomaly detection in iot sensors in iot sites using machine learning approaches," *Internet of Things*, vol. 7, p. 100059, 2019.
[36] M. Shafiq, Z. Tian, Y. Sun, X. Du, and M. Guizani, "Selection of effective machine learning algorithm and bot-iot attacks traffic identification for internet of things in smart city," *Future Generation Computer Systems*, vol. 107, pp. 433–442, 2020.
[37] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
[38] W. Haider, N. Moustafa, M. Keshk, A. Fernandez, K.-K. R. Choo, and A. Wahab, "Fgmc-hads: Fuzzy gaussian mixture-based correntropy models for detecting zero-day attacks from linux systems," *Computers & Security*, p. 101906, 2020.
[39] Y. N. Soe, Y. Feng, P. I. Santosa, R. Hartanto, and K. Sakurai, "Towards a lightweight detection system for cyber attacks in the iot environment using corresponding features," *Electronics*, vol. 9, no. 1, p. 144, 2020.
[40] B. Venkatesh and J. Anuradha, "A review of feature selection and its methods," *Cybernetics and Information Technologies*, vol. 19, no. 1, pp. 3–26, 2019.
[41] "An effective feature engineering for dnn using hybrid pca-gwo for intrusion detection in iomt architecture," *Computer Communications*, vol. 160, pp. 139 – 149, 2020.
[42] M. Rahimi, M. Songhorabadi, and M. H. Kashani, "Fog-based smart homes: A systematic review," *Journal of Network and Computer Applications*, vol. 153, p. 102531, 2020.

Prabhat Kumar is working towards his Ph.D. degree in Information Technology, National Institute of Technology, Raipur, India. He earned his Ph.D. scholarship position as a talented student. He has over 10 publications in high-ranked Journals and Conferences. His research interests are the Internet of Things, Software-defined Networking, privacy-preservation techniques, intrusion detection systems, malware detection systems, Machine learning, Deep learning and Blockchain. He is also an IEEE Student Member.

Randhir Kumar is working towards a Ph.D. degree in the Department of Information Technology, National Institute of Technology, Raipur. He has published more than 20 research articles in the areas of Blockchain Technology and Its Framework. His research interests include Blockchain Technology, Cryptography Techniques, Information Security, Web Mining, and Image Processing. He is also an IEEE Student Member.

Gautam Srivastava is an Associate Professor at Brandon University, Canada. In his 8-year academic career, he has published a total of 200 papers in high-impact conferences in many countries and high-status journals (SCI, SCIE). He is an Editor of several international scientific research journals. He received his M.Sc. and Ph.D. in Computer Science from the University of Victoria, Canada.

Govind P. Gupta received his Ph.D. degree from the Indian Institute of Technology, Roorkee, India, in 2014. He is currently an Assistant Professor in the Department of Information Technology at the National Institute of Technology, Raipur, India. His current research interests include efficient protocol design for Wireless Sensor Networks and Internet of Things, Network Security and Software-defined Networking. He is a professional member of the IEEE and ACM.

Rakesh Tripathi received his Ph.D. degree in Computer Science and Engineering from the Indian Institute of Technology Guwahati, India. He is an Assistant Professor with the Department of Information Technology, National Institute of Technology, Raipur, India. He has over ten years of experience in academics. He has published over 20 referred articles and served as a reviewer of several journals. His research interests include Mobile-Adhoc Networks, Sensor Networks, Data Center Networks, Distributed Systems, Network Security, Blockchain and Game Theory in Networks. He is also an IEEE Senior Member.

Thippa Reddy G is currently working as an Associate Professor in School of Information Technology and Engineering, VIT, India. He obtained his Bachelor of Technology degree in Computer Science and Engineering from Nagarjuna University, Andhra Pradesh, India, Master of Engineering in Computer Science and Engineering from Anna University, India and completed his Ph.D. in Engineering from Vellore Institute of Technology, India. He published more than 80 papers in reputed SCI/SCIE journals. Currently, his research interests include Machine Learning, Deep Learning, Computer Vision, Big Data Analytics, Blockchain.

Neal N. Xiong (S'05–M'08–SM'12) is current an Associate Professor (5rd year) at Department of Mathematics and Computer Science, Northeastern State University, OK, USA. He received his both Ph.D. degrees in Wuhan University (2007, about Sensor System Engineering), and Japan Advanced Institute of Science and Technology (2008, about dependable communication networks), respectively. His research interests include Cloud Computing, Security and Dependability, Parallel and Distributed Computing, Networks, and Optimization Theory.